With a table approach, the memory size is minimal, once the interpreting program space is considered. A change in the FSA requires only a table change and a few changes in the affected semantic procedures, nothing else. An automatic FSA generator should clearly generate reduced sparse tables that can be immediately used in a general-purpose FSA interpreting program.

Exercise

Propose other program-oriented FSA representations, including one in assembly language for your favorite machine architecture.

3.7. Applications of FSA

FSA have many applications in compilers. The parser in an LR(1) recognizer contains a FSA (section 2.3.4). Certain symbol table operations are nicely modeled as a FSA (section 7.4). We shall explore some of the concepts of lexical analysis in the remainder of this chapter.

3.7.1. Recognition of Literals

Most common programming languages contain a class of literal constants that can be recognized by a finite-state machine. They may appear in the compiler source language and in data to be formatted.

Let us consider one example, based on the automaton studied in the first part of this chapter. The design of a recognizer may start with a deterministic or nondeterministic automaton, a regular expression or a regular grammar. The choice depends on the nature of the literal specification in the language. If the specification is in BNF, it may be best to attempt to construct a regular expression from the BNF by the expansion methods of section 3.5. In any case, the end result of a definition and a reduction must be a deterministic FSA, such as the one in figure 3.1 for simple decimal numbers.

Let us add conversion operations to this machine. In general, we assign one or more registers to the machine, and then assign operations on the register contents to each of the transitions. The result is a machine as shown in figure 3.27. There are three registers, SIGN, P, and N. SIGN holds either -1 or $+1$. The registers P and N hold floating-point numbers. Certain of the transitions carry operations on these registers. We include transitions into the start state and out of the halt states for completeness.

Initially, SIGN is $+1$, P is 10.0, and N is 0.0. SIGN is changed to -1 only upon the transition S to A on "$-$"; otherwise it remains $+1$. Each transition

on a digit "d" involves the function "value(d)"—the numeric equivalent of the character "d". Thus in the S to B transition, value(d) is assigned to N. Upon an exit from B or H, the final value of the number is SIGN*N.

On the transition B to B, the old value of N is multiplied by 10 and added to value(d). The digit d precedes any decimal point, hence each digit causes the place value of those preceding them to be increased by a factor of 10.

On scanning a decimal point, the register P is used; P carries the power of ten corresponding to the place value of the digits following a decimal point. Thus in the transition G to H, value(d)/P is added to N, and P is increased by a factor of 10.

An exit from halt state B clearly means that no decimal point was scanned; the number is an integer. The exit from halt state H means the number carries a decimal point.

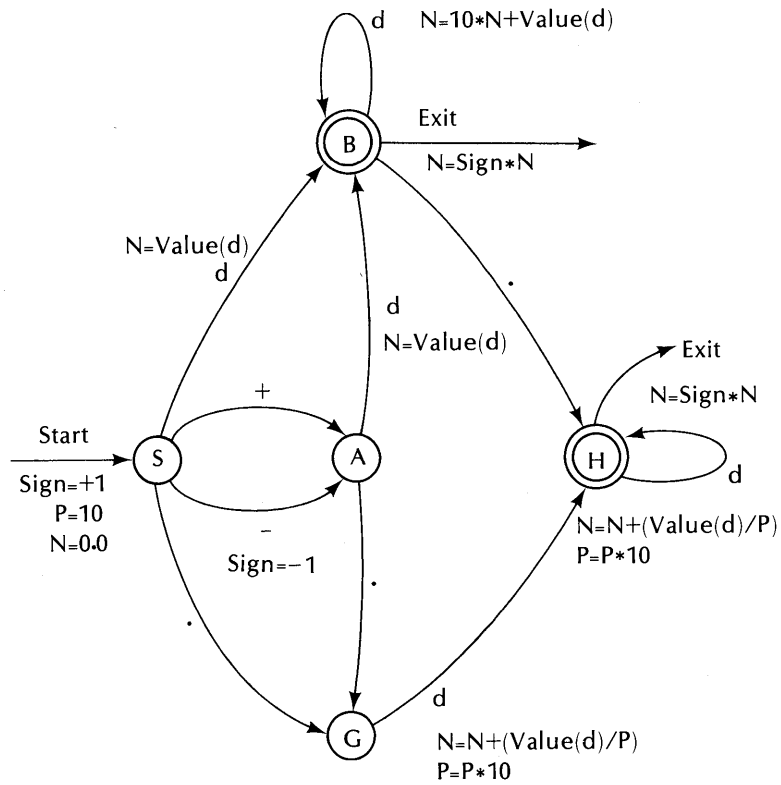

Figure 3.27. The machine of figure 3.1 with semantic operations added. This machine recognizes a decimal number and converts it to an internal form.

The exits from B and H are clearly taken on any token other than "." or "d" for B or "d" for H. A *lookahead* of one token is evidently needed in order to exit properly. However, there may be other kinds of halt indicator. If the recognizer is used in a Fortran formatter to recognize numbers in an input record with fixed column boundaries, it may be necessary to halt on the end of the field, not the next character. Such details can clearly be worked out as a programming exercise.

Exercises

1. Trace figure 3.27 with the string "−33.62".
2. Extend figure 3.27 to exponent parts for floating point numbers.
3. Discuss optional blanks in a language: (a) If a number can be preceded by any number of blanks, what additional states are needed in the FSA of figure 3.27? (b) If a number can contain blanks separating any of its parts except the digits in the numbers, what additional states and/or transitions are needed?
4. Construct a FSA that accepts either numbers, Fortran identifiers, quoted strings, or comments. The quoted strings are delimited by a pair of quote marks, and may contain a quote mark by writing it twice. A comment begins with the keyword COMMENT and ends on a semicolon. Invent something to deal with the problem that COMMENT is in the class of Fortran identifiers.
5. Make an assertion that holds for each of the states in the FSA of figure 3.27. Use these states to prove that the FSA correctly generates the numeric equivalent of accepted input strings. For example, a reasonable assertion for any of the states is, "N is the absolute value of the decimal number scanned so far."

3.7.2. Lexical Analysis

A *lexical analyzer* assembles sequential groups of one or more characters in the source into tokens. The process may be relatively simple or quite difficult, depending on the language.

A lexical analyzer comprises a number of functions:

1. Source file handler. The lexical analyzer is responsible for opening and reading the source file. Some compilers permit several input files to be merged, or for a source file to call out another file, etc. Record

boundaries must either be made into tokens or made invisible to the parser.

2. Comment scanner. Most programming languages contain provisions for comments, which may appear anywhere and are therefore difficult to build into a grammar. The lexical analyzer must then recognize them and make them invisible to the parser.

3. Macro processor. A macro processor is essentially a string expander that looks for macro definitions and macro calls and expands the latter. Usually, the macro processor has little or no relation to the syntax of the source program that it generates, and it may therefore process the source independently of the rest of the compiler.

4. Token assembler and screener. The lexical analyzer is responsible for collecting sequences of characters in the source into identifiable tokens for the parser. The tokens it assembles correspond to the terminals of the language's grammar. The recognition of tokens may not be a trivial matter, as some programming languages contain keywords that can also be used as variable names.

5. Literal converter. Floating point literals, fixed point literals, literal strings, and other literal forms may all be recognized and converted to internal form by the lexical analyzer.

Source Records and Characters

A programming language may be either free-form or line-oriented. Algol 60, Algol 68, Pascal, and PL/I are examples of free-form languages; Basic and Fortran are examples of line-oriented languages.

In a free-form language, the characters comprising the source are conceptually assumed to be one long sequence uninterrupted by source record boundaries. Thus if the physical records of the source happen to be 80 characters each, some procedure in the compiler must arrange that the 80th character of each record is effectively contiguous with the first character of the next record; that is, the record boundaries must be invisible to the lexical analyzer and the parser. The lexical analyzer is best organized in two procedural levels: GETCHAR and GETTOKEN, as follows:

GETCHAR is at the lowest level, is called by GETTOKEN, and supplies one character on each call. It is responsible for accessing the source file and for processing any special control records that are not in the compiler's language. For example, in many large systems, records with certain special characters in the left-most columns must be recognized as control records, with a special syntax.

GETTOKEN calls GETCHAR for each character needed in assembling a token. It knows nothing of the source record arrangement, but must be tailored to the language. GETTOKEN is called by the parser system and supplies one token for each call.

In a line-oriented language, the parser and scanner system may be reset to some known starting state at the beginning of each line. Line boundaries may or may not serve as delimiters, however, the beginning of the next line certainly serves as an end-of-line delimiter. The Fortran conventions are interesting and illustrate a class of lexical analyzer problems. The lines are organized as 80-byte records, corresponding to a standard IBM card. Column 1 is used to indicate that the record is a comment. A statement label number may appear in columns 2 to 5. Column 6 is not used on the first of several continuation records. The source text resides in columns 7 to 72. If column 6 of the next record is marked, it serves as a continuation record, in which case, column 7 of the second record must effectively follow column 72 of the first (the boundary is invisible). Up to 19 continuation records are permitted. Statement labels are not permitted on continuation records.

A Fortran statement therefore has the form

<label> <delimiter> <statement> <end-of-statement>

where <delimiter> and <end-of-statement> are special tokens that separate the statement label, the statement, and the next record.

A *token* is a member of the terminal alphabet of the grammar and may be a single character or one of the following common character sequences:

Two or more special characters, e.g., {:=, <=, **}.

Identifier. Generally an identifier begins with a letter and continues with letters, digits and (sometimes) certain special characters, e.g., PAY_ROLL (Cobol), or S156 (most common languages). Identifiers must usually be classified as *user names* and *keywords*. A user name is invented by the programmer to serve as a data, procedure, or statement label. A keyword is a terminal token in the language's grammar that happens also to fit an identifier pattern. In Pascal, WHILE, DO, IF, THEN, etc. are keywords. The task of separating keywords and user names is often difficult.

Number. The number formats of the language often include fixed- and floating-point forms, e.g., -17.56E-22 (Fortran). In most compilers, the lexical analyzer recognizes and converts such numbers. However, a complicated number format may also be easier to recognize through the parser system, so that the lexical analyzer need only return the smaller tokens "17", ".", "E", etc.

Quoted string. Many programming languages permit an arbitrary sequence of characters to be quoted and treated as a single token, a *string*.

Counted string. One of the Fortran FORMAT statement forms is a string preceded by a count, e.g., 5H# #)ab. The string following the "H" can only be determined by first scanning the count "5" and then counting off five characters; this is a lexical analyzer task.

Distinguishing Tokens

Algol 60 contains each of the following tokens:

: used between a statement label and the statement.
= used as an "equal" relation.
:= used as the replacement operator.

The lexical analyzer must distinguish these, and can do so by a left-to-right scan of the source characters, along with a single character lookahead. If ":" has been scanned, and the next character is not "=", then the returned token must be ":". In general, the analyzer should assemble the longest sequence of characters that is consistent with a single token in the language. Whether this rule will be effective must be investigated through a study of the grammar. The grammar should be such that token "=" can never follow ":" except in the composite token ":=". Other composite tokens must satisfy a similar property. A discussion of the *follow* problem is given in the next chapter.

A *delimiter* is any character that serves to terminate a token; it is not a part of the terminated token, but may itself be a token or part of a token. In some languages, a blank is a valid delimiter. In others, Fortran in particular, a blank is ignored except in quoted strings and therefore cannot serve as a delimiter.

If blanks are everywhere ignored, then tokens must be delimited by other tokens. Consider this example in Fortran:

DO10I = 1 7 E 3 . G T . X

We have separated the characters of the statement for clarity. It happens that the lexical analyzer cannot determine whether this statement begins with the keyword "DO" or the identifier "DO10I" until the "E" is scanned. A DO statement would have to have another digit or a comma before the "E" position. (This example draws upon the Fortran rule that a DO loop variable must be fixed-point. If, in some Fortran extension, a DO loop variable could be floating-point, then the lexical analyzer would have to scan to the first "." before determining that the statement is an assignment.) On the other hand, if one or more blanks were required as delimiters of tokens, but not permitted within a token, the statement would have to read

```
DO10I = 17E3 . GT . X
```

A DO statement would then have to carry a separating blank between the DO and the next token, e.g.,

```
DO 10I=17,1,25
```

Other blanks are unnecessary, since the remaining tokens can be distinguished.

There are several distinct language policies regarding the use of blanks and reserved words, as follows:

1. No policy, as in Algol 60. The problem of distinguishing keywords and separating tokens is left to an implementation. With no policy, a program may be untransportable without extensive editing.

2. Blanks must be inserted as needed as separators, and keywords are reserved. Many implementations follow this policy, for the sake of simplicity of the compiler.

3. Blanks must be inserted, and keywords are not reserved. (e.g., PL/I).

4. Blanks are ignored, ala Fortran, and keywords are reserved. (e.g., ANS Fortran subset).

5. Blanks are ignored, and keywords are not reserved. (e.g., ANS full Fortran).

One of these policies usually applies to a language to be implemented. Of these, policy 2 is the easiest. The analyzer can easily distinguish tokens in a left-to-right scan, and keywords can be distinguished from user identifiers through a table lookup. Unfortunately, this policy can be vexing to a compiler user. For example, PL/I contains over 100 keywords. It is unreasonable to expect every user to know every one of these keywords and never declare any of them as an identifier. A policy 2 implementation will also yield obscure syntax errors unless great care is taken to look for and properly report errors stemming from the use of keywords as identifiers.

Policy 3 is somewhat more difficult to implement, assuming the language is unambiguous under the policy. With inserted blanks, tokens are easily distinguished. However, the compiler may have to scan several tokens into the source string on some assumption regarding an identifier before finding that the assumption was right or wrong. If wrong, it must backtrack and try an alternative. Keywords are rarely used as identifiers, hence if the first choice is always "keyword" (if a choice exists), the amount of backtracking required will be small.

Policy 3 is considerably aided by a language policy of predeclared variables. Then the analyzer can be alert to any keywords that have been declared in its backtracking process. A keyword that has not been previously declared can be positively identified as such and not confused with an identifier.

Even this procedure breaks down if a variable declaration can begin with a declared identifier. Fortunately, most common programming languages require some keyword in a declaration preceding the declared identifier. Thus in Fortran, a declaration is triggered by one of the keywords COMMON, DIMENSION, REAL, INTEGER, etc. In PL/I, a declaration is triggered by the keyword DECLARE or DCL, and the declaration must appear first in a block. In Pascal, a declaration is triggered by one of the keywords TYPE or VAR.

Now consider an identifier DCL in PL/I declared in some block. The compiler will evidently have a problem when DCL is encountered at the head of some inner block. Is this a declaration or some statement beginning with DCL? For example,

```
BEGIN
    DCL  DCL FIXED BINARY;
         .
         .
         .
    BEGIN
      DCL I FIXED BINARY;  /* This DCL is a problem */
      DCL := I;     /* So is this DCL */
      DCL := I-1;     /* This DCL is not a problem */
         .
         .
         .
    END;
END;
```

Once the compiler is into the executable statements of a block, a declaration is no longer permissable, and DCL must be the user identifier. Other keywords can be distinguished by the keyword marking technique discussed above, based on the previously declared identifiers.

Policies 4 and 5 are still more difficult. The lexical analyzer must now make some decisions based on substrings of statements. Without certain other restrictions in the language, lexical analysis of a policy 5 language may be impossible, and the language may in fact be ambiguous—all this despite the existence of a perfectly reasonable and unambiguous high-level grammar for the language.

An example of a policy 5 language is Fortran. What makes Fortran translatable and unambiguous are other properties, as follows:

1. A statement is bounded in length and has a well-defined origin. These are defined by special record and continuation conventions (described above). A reasonable analyzer strategy is then to first bring a complete statement into a string buffer (the maximum size required is 1360 characters—20 maximum lines at 68 characters per line), so that the scanning can backtrack as needed without requiring the source file handler to back up.

2. Each statement either is a replacement statement of the form

<name> <optional index> = <expr>

or a control statement with a keyword prefix, e.g.,

<keyword> <remainder of control statement>

3. Names are limited to seven characters.

These help limit the number of possibilities in a statement. Unfortunately, ANS Fortran permits a variable declaration to appear anywhere in the program, and variables need not be declared. This means that any statement, including the last one, is potentially a statement beginning with a user identifier masquerading as a keyword.

Now consider the replacement statement. If the <optional index> is present, some Fortran implementations permit an arbitrary expression for the index, e.g.,

X(A+B*(17−S)) = Y

ANS Fortran permits only simple expressions of the form A*X± B as an array index. An expression index could continue through most of the 1360 characters of the statement buffer. Also the left part of a replacement has the same form as the left part of an IF statement:

IF(<expr>) <number>,<number>,<number>

or a FORMAT statement:

FORMAT(<format specification>)

An IF statement cannot be distinguished from an indexed replacement until the character past the closing right parenthesis is scanned.

A FORMAT can also create difficulties for a lexical analyzer. For example, suppose the strategy is to match left and right parentheses in an attempt to locate the closing right parenthesis of an indexed replacement, ignoring other context. This appears on the surface to be reasonable. Unfortunately, this strategy fails on certain Fortran statements, for example:

FORMAT(5H223)=, I5, 4X)

Such a scan will continue past the replacement symbol "=", and falter on the comma. The difficulty, of course, lies in the failure to fully parse a replacement syntax (the "5H223" doesn't belong as an index).

A backtracking strategy can deal with all the special problems of Fortran and other languages with difficult lexical policies. While backtracking, nothing much else can be done, since it is possible that any additional work must be undone later. However, it may be possible to build the abstract syntax tree for the statement, with no other immediate action. The initial parsing assumption is that the statement is a replacement. When it is apparent that this assumption is wrong, it may be possible to make some minor changes in the tree to reflect the alternative, and continue parsing. This approach requires that the parser be sufficiently general to accept both kinds of statement and that the parse trees for the alternatives are not too different. The symbol table should not be changed until the possibility of a wrong choice is eliminated.

Comments and Quoted Strings

A comment in Algol 60 begins with the reserved word COMMENT and ends on the next semicolon. Other languages have a similar arrangement. In Pascal, a comment is enclosed in braces { }, and in PL/I a comment is enclosed in the character pairs /* and */.

A quoted string is used as a data element in many languages, and is delimited by a pair of quotation marks. The beginning and ending quotation marks are sometimes the same character and sometimes different characters. In any case, the rule for quoted strings usually is that the string begins after the first quotation mark and continues to the terminating quotation, through whatever characters appear—blanks, comment strings, etc.

In dealing with comments and quoted strings, the lexical analyzer must have three states: an *outer* state S in which it looks for language tokens, a *quote* state Q in which it is scanning a quoted string, and a *comment* state C in which it is scanning a comment string. In state S, it may transfer to state Q on a quote mark or to state C on a comment opener. It remains in S for all other tokens. While in state C, the source is scanned and the analyzer should be sensitive to only two tokens—an end-of-file and a closing comment token. All others are ignored. The closing comment token causes a return to state S; an end-of-file should cause an error message and an orderly termination of the compile.

State Q is similar—only a closing quotation mark or an end-of-file is of interest.

Exercises

1. Outline a lexical analyzer for Algol 60. Start by making a list of the

tokens in the language, then examine the rules regarding reserved identifiers, blanks, and comments. Finally, design a finite-state automaton that will classify the tokens. Consider identifiers, quoted strings, and numbers as tokens.

2. Discuss the problem of efficient recognition of reserved words. For example, does it make sense to walk down through a tree on a sequence of letters, such that upon reaching some terminal node, the identifier ends? (You may wish to read chapter 6 next, or come back to this problem after studying chapter 6.)

3.8. Some FSA Theorems and Their Proofs

In this section, we prove a number of theorems stated earlier in the text.

3.8.1. Equivalence of Empty Cycle States

Let p and q be two states in a machine M such that $p \vdash^+ q$ and $q \vdash^+ p$ on empty moves only. Let x be any string accepted by M starting from state p (but p need not be the start state).

Then $p \vdash^+ q$ implies that there exists a sequence of states $p_1, p_2, \ldots p_n$ $(n \geq 0)$ such that

p_1 is in $\delta(p, \epsilon)$,
p_2 is in $\delta(p_1, \epsilon)$,
.
.
.
p_n is in $\delta(p_{n-1}, \epsilon)$, and
q is in $\delta(p_n, \epsilon)$

But then $(p, x) \vdash (p_1, x) \vdash (p_2, x) \vdash \ldots \vdash (p_n, x) \vdash (q,x)$ and therefore x is accepted by M starting from state q.

If we interchange "p" and "q" in this argument, we can show that any state x accepted by M in state q is also accepted by M in state p, therefore states p and q must be equivalent. QED.

3.8.2. Equivalence Through Removal of Empty Moves

A simple algorithm for the removal of empty transitions from a machine M to yield a machine M′ was given in section 3.2.2. We have already shown that the algorithm terminates (the absence of empty cycles is crucial). It should be clear that no empty moves remain. We need to show equivalence. We do this by proving the somewhat stronger lemma:

Lemma. For all (x in Σ^* , p in Q) ($(p, x) \vdash^* (f, \epsilon)$ in M if and only if $(p', x) \vdash^* (f', \epsilon)$ in M') where f is in F and f' is in F'.

For convenience, we indicate the states in M' with primed letters, e.g., p', q', and the states in M with unprimed letters, e.g., p, q. Q is the state set in M, and F is the set of halt states in M. The algorithm does not remove or add any states, so that every state p in M corresponds to a state p' in M'.

Proof: "Only-if" part. Let $(p, x) \vdash^* (f, \epsilon)$ in k moves, $k \geq 0$.

The basis step (k = 0). Clearly p = f and is in F. But p' must be in F' by state correspondence.

The inductive step (k > 0). Consider the first step in the machine move sequence:

$$(p, ax) \vdash (q, x) \vdash^* (f, \epsilon), \text{ where a is in } \Sigma \cup \{\epsilon\}$$

We have several cases to consider. If a is nonempty, then q' is in $\delta(p',a)$ and therefore $(p',ax) \vdash (q',x)$. But then (q',x) must yield (f, ϵ) in k-1 moves or less, by the inductive hypothesis, hence M' accepts ax in state p'.

If a is empty, we have $(p, x) \vdash (q, x) \vdash^* (f, \epsilon)$, with q in $\delta(p, \epsilon)$. But q' is not in $\delta(p', \epsilon)$ in M'. We now have two subcases to consider: x empty or not.

If x is empty, then $(p, \epsilon) \vdash (q, \epsilon) \vdash^* (f, \epsilon)$. The complete sequence involves only empty moves, with a sequence of states $(p, q, q_1, q_2, \ldots, f)$. By the halt state rule in the construction process, each of the states p', q', $q'_1, \ldots$ must be halt states. But then (p', ϵ) is an accepting configuration for M'.

If x is nonempty, there must be some first nonempty move in the move sequence, as follows:

$$(p, ax) \vdash (p_1, ax) \vdash \ldots \vdash (p_n, ax) \vdash (q, x) \vdash^* (f, \epsilon)$$

But by the construction process, we must have

q' in $\delta(p'_n, a)$,
p'_{n-2} in $\delta(q',a)$,
.
.
.
p' in $\delta(q',a)$.

Hence, $(p', ax) \vdash (q', x) \vdash^* (f', \epsilon)$. (The latter set of moves follows from the inductive hypothesis; there are less than k moves in that sequence). QED

A proof of the "if" part is similar in character.

3.8.3. Equivalence on the NDFSA to DFSA transformation

We show that L(M) is a subset of L(M'), where M' is derived from M by the NDFSA → DFSA transformation of section 3.2.3.

Proof: Let $(p, x) \vdash^* (f, \epsilon)$ in M in k steps. We show that $([\ldots p\ldots], x) \vdash^* ([\ldots f\ldots], \epsilon)$ in M', for every state $[\ldots p\ldots]$ in M'.

Basis, k = 0: Here p = f and f is in F. But then $[\ldots p\ldots]$ must be in F', hence x is accepted by M.

Inductive step, k > 0: Consider the first step

$$(p, ax) \vdash (q, x) \vdash^* (f, \epsilon)$$

where "a" is in the alphabet. (*Note*: No empty moves can exist). But then we have q in $\delta(p, a)$ and therefore $[\ldots q\ldots]$ is in $\delta([\ldots p\ldots], a)$ in M' for every state of the form $[\ldots p\ldots]$ in M'. Then

$$([\ldots p\ldots], ax) \vdash ([\ldots q\ldots], x) \vdash^* ([\ldots f\ldots], \epsilon)$$

The proof that L(M') is a subset of L(M) is similar. QED

3.8.4. The Pairs Table Reduction Algorithm

We divide the proof into a lemma and a theorem. The theorem applies to the pairs table upon completing its construction, and the lemma to the completion of the marking process.

Lemma. The feasible state-pairs contain all the equivalent state-pairs of the FSA.

Theorem. The unmarked state-pairs contain all and only the equivalent state-pairs of the FSA.

A proof of the lemma is left as an exercise. A proof of the theorem follows.

Proof: The "All" Part. Let (p, q) be an equivalent state-pair in the FSA. Then by the lemma, it is among the feasible state-pairs in the construction of the pairs table. Now suppose (p, q) becomes marked during the marking phase, and therefore is not among the unmarked set. It became marked because some token caused a transition from it to some marked or absent state-pair, (p', q'). If (p', q') were absent, then states p' and q' must be distinguishable, by the lemma, and therefore (p, q) are distinguishable states. This statement contradicts the assertion of their equivalence. Suppose instead that the state-pair (p', q') is marked. By a repetition of this argument, there must have

been a transition from it to some other state-pair that caused it to be marked. The first such marking in this chain had to be caused by a transition to a missing state-pair. Hence (p′, q′) is distinguishable, and (p, q) must be distinguishable, which again contradicts the assertion of equivalence.

Proof: The "Only" Part. Let (p, q) be an unmarked pair appearing in the pairs table at the end of the process, but p and q are distinguishable in the machine. That they appear in the feasible set is no evidence of equivalence or distinguishability. However, if they are distinguishable, then there is some string $x = a_1a_2a_3 \ldots a_n$ such that x is accepted by the machine in state p, but not in q, or vice versa. Without loss of generality, assume that q is the non-accepting state. Now the failure to accept can be the result of a machine block prior to completing string x, or the result of completing x, but terminating in a non-halt state. Suppose first that the machine blocks. Then the state sequences beginning with p and with q look like this:

$$p \vdash p_2 \vdash p_3 \vdash \ldots \vdash p_n \qquad \text{(acceptance)}$$

$$q \vdash q_2 \vdash \ldots \vdash q_m \qquad \text{(block)}$$

where $m < n$, and $n = |x|$. Now states p_m and q_m are clearly not in the feasible state-pair set, since at least one transition (on the mth token of string x) can occur on p_m, but not on q_m. It follows that (p_{m-1}, q_{m-1}) is either not in the feasible state-pair or has become marked, since there is a transition from the pair (p_{m-1}, q_{m-1}) to (p_m, q_m), and the latter is not in the feasible state-pair set. Similarly, (p_{m-2}, q_{m-2}) become marked, etc., and eventually (p, q) become marked.

If the failure to recognize string x in state q is because of a nonhalt state upon completing x, then the state sequences beginning with p and with q look like this:

$$p \vdash p_2 \vdash p_3 \vdash \ldots \vdash p_n$$

$$q \vdash q_2 \vdash q_3 \vdash \ldots \vdash q_n$$

Here, p_n is in the halt set, but q_n is not. The pair (p_n, q_n) therefore cannot be in the feasible state-pair set. Thus (p_{n-1}, q_{n-1}) become marked, and because they are marked and there is a transition step from (p_{n-2}, q_{n-2}) to (p_{n-1}, q_{n-1}), the pair (p_{n-2}, q_{n-2}) become marked, etc. Eventually (p, q) becomes marked. QED

3.9. Bibliographical Notes

The earliest work on FSA is in McCullough [1943]. Kleene [1952] and [1956] first introduced the notion of regular expressions. Algorithms for the

interconversion of state transition functions and regular expressions are found in McNaughton and Yamada (McNaughton [1960]). An early review paper is Brzozowski [1962]. The material on equivalence is largely from Huffman [1954], Moore [1956], Mealy [1955], and Aufenkamp and Hohn (Aufenkamp [1957]), as found in Gill [1962]. A regularity test for a context-free grammar is given by Stearns [1967]. The literature on FSA and their applications to logic circuitry is very large; some representative texts are Booth [1967], Gill [1962], Harrison [1965], Hartmanis and Stearns (Hartmanis [1966]), Kohavi [1971], Maley and Earle (Maley [1963]), McCluskey [1965a], and McCluskey and Bartee (McCluskey [1965b]). In addition, Aho and Ullman (Aho [1972a] and Hopcroft and Ullman (Hopcroft [1969]) contain material on finite-state automata and their relation to language recognition. Lexical analysis has been discussed by many authors; a representative sample is Johnson [1968], Conway [1963], DeRemer [1974c], Gries [1971], Feldman [1968].

CHAPTER 4

TOP-DOWN PARSING

A top-down parser conceptually develops a derivation tree for a sentence in the language from the root node down. We have seen previously that the essential problem is that of deciding which of several productions with the same left-member applies in the tree next. We always know the next left-member, for it is associated with a tree node known to be a part of the final derivation tree.

We first address ourselves to the general problem of a top-down translator for a context-free language by examining a general nondeterministic parser. Although a nondeterministic machine is impractical in a real compiler, it exhibits many of the properties that must exist in a real compiler. Furthermore, any context-free language can be parsed by such a machine and we can easily prove that the machine recognizes exactly the language of the context-free grammar used to construct it.

A deterministic top-down parsing machine unfortunately cannot be constructed for every context-free grammar; there are context-free languages which cannot be recognized by a deterministic top-down parsing machine.

However, those grammars with a deterministic top-down parser are sufficiently powerful to define many common programming languages and therefore to construct useful and efficient compilers.

4.1. Nondeterministic Push-Down Automata

The nondeterministic top-down parser we shall construct is an example of a general class of push-down automata, or PDA in short. A *push-down*, or *stack automaton*, contains (1) an *input string*, of symbols in the alphabet of the machine, (2) a *read head*, which may examine one symbol in the input list at a time and may move only from left to right, (3) a *finite-state machine*, which serves to control the system's operations, and (4) a *last-in-first-out push-down stack*. See figure 4.1(a).

A *move* in a PDA is governed by the present state, the input symbol under the read head, and the symbol on the top of the push-down stack. In a move, the read head is advanced, the state is changed, and the top stack symbol is replaced by some string (possibly empty).

A PDA scans its input string by a succession of such moves. It may be unable to move at some point, in which case some other set of choices made earlier must be tried. If a set of choices permits the machine to scan the input

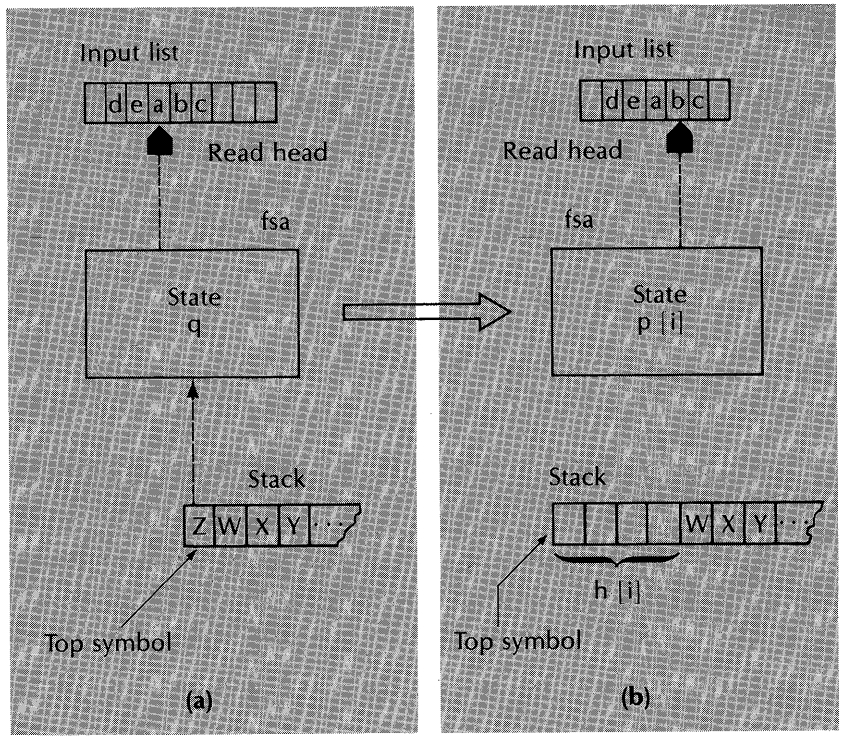

Figure 4.1. Non-empty move of a push-down automaton. The read head is advanced one token in the input list, the state changes from q to p[i], token Z on the stack is replaced by a string h[i].

string and either end in a halt state, or empty its stack, then the input string is said to be *accepted* by the PDA.

A PDA is therefore a 7-tuple (K, Σ, H, δ, q_0, Z_0, F), where:

- K is a finite set of states.
- Σ is a finite input alphabet, i.e., the set of symbols appearing in the input list.
- H is a finite push-down stack alphabet. For generality, we permit different symbols to be used on the push-down stack than elsewhere.
- $q_0 \in K$ is the initial state of the machine.
- $Z_0 \in H$ is the initial symbol on the push-down stack.
- $F \subset K$ is a set of final or halt states.
- δ is a transition function, mapping a triple (q, a, Z) to a set of pairs $\{(p_1, h_1), (p_2, h_2), \ldots\}$, where q is in the state set K, "a" is a member of $\Sigma \cup \{\epsilon\}$, Z is in the push-down stack alphabet H, $p_1, p_2, \ldots$ are members of the state set K, and $h_1, h_2, \ldots$ are strings in H^* .

A PDA move is governed by the transition function δ, just as in a finite-state automaton. However, in a push-down automaton, the move is controlled not only by the next input symbol and the present state but also by the symbol on the top of the push-down stack. Furthermore, the result of a move is not only some new state, but also the replacement of the top symbol on the push-down stack by a string of symbols drawn from the stack alphabet H.

Let us explore a typical move in more detail. Consider a member of the transition function:

$$\delta(q, a, Z) = \{(p_1, h_1), (p_2, h_2), \ldots (p_m, h_m)\} \tag{4.1}$$

In figure 4.1(a), the PDA is shown in a state in which symbol "a" is under the read head, the controlling FSA is in state q, and the top symbol on the push-down stack is Z. These are the conditions necessary to invoke the move expressed by the transition function member, Eq. (4.1) above. The move is made by choosing one of the pairs in the set $\delta(q, a, Z)$, assuming that at least one pair exists. Suppose we choose the pair (p_i, h_i). Then the next state of the FSA is p_i, the symbol Z on the stack top is replaced by the string h_i, and the read head is advanced to the next symbol in the input list, figure 4.1(b). (Note that the push-down stack is drawn with its top to the left, which is useful in describing a top-down PDA, as we shall see.)

There are two possible variations on Eq. (4.1). The symbol "a" in $\delta(q, a, Z)$ may be the empty string, ϵ. If this is so, then the machine may move without considering the input symbol; it may move on the basis of its state and the top stack symbol alone. The move is also made without moving the read head.

In a second variation, the string h_i in a transition set pair may be empty. Then the stack top symbol Z is effectively popped from the stack, exposing the symbol (if any) beneath it. With this variation, the stack can be reduced and ultimately emptied.

A PDA may halt in either of two ways—by empty stack or by final state. It is said to halt by empty stack and accept the input string if, upon the move in which the read head advances just past the end of the input list, the push-down stack is emptied. It is said to halt by final state and accept the input string if, upon the move in which the read head advances just past the end of the input list, the FSA enters a member of the final state set F. A given PDA is always defined to halt in one manner or the other. However, it can be shown that for every PDA of one kind, there exists an equivalent PDA of the other kind, so that we may choose whichever is more convenient.

In either case, a nondeterministic PDA is said to accept a given input string if there exists a sequence of moves that lead from its initial state and stack contents to a halt condition.

The PDA is said to *block* if it fails to accept an input string.

Configurations and moves

A *configuration* of a PDA is a triple (q, w, h), where q is some state in K, w is the portion of the input list from the read head position to its right-end and a member of Σ^*, and h is the contents of the push-down stack and a member of H^*.

A configuration contains all the information needed to predict the future behavior of a PDA. For example, the PDA of figure 4.1(a) may be described by the configuration

$$(q, abc\ldots, ZWXY\ldots)$$

The PDA of figure 4.1(b) may be described by the configuration

$$(p_i, bc\ldots, h_iWXY\ldots)$$

A move in a PDA may be regarded as a means of transforming one configuration into another one, which provides a more concise way of defining a machine move, as follows.

We say that

$$(q, ax, Zh') \vdash (p, x, hh')$$

is a possible move if and only if (p, h) is in $\delta(q, a, Z)$. (Note that this move definition is consistent with the possibility that $a = \epsilon$, and $h = \epsilon$.)

A sequence of one or more moves is denoted by $\vdash^+$. A sequence of zero or more moves, where a "zero" move is no change in the present configuration, is denoted by $\vdash^*$. With this notation, we may define the language L(M) recognized by a PDA M as:

$$L(M) = \{ w \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon) \} \tag{4.2}$$

for some p in K, where machine M halts by empty stack, or as:

$$L(M) = \{ w \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, h), p \text{ in } F \} \tag{4.3}$$

for some h in H^*, where machine M halts by final state.

The definition in (4.2) may be translated to English roughly as follows: The language L(M) of a PDA M consists of all those strings w such that, given a PDA initially in state q_0, with stack contents Z_0, and the read head positioned at the first symbol of w, there exists a sequence of moves that results in the read head completing the input string, an empty stack, and some end state (not necessarily a final state).

The definition in (4.3) may be similarly interpreted.

Example Consider the two-state machine defined by figure 4.2. The states are P and Q, the input symbols are 0 and 1, and the stack symbols are R, B, and G. This machine recognizes all palindromes in the symbols 0 and 1, i.e., the language

	"From"			"To"			
Row	State p	Input symbol a	Stack symbol z	State q_1	Stack h_1	State q_2	State h_2
1	P	0	R	P	BR		
2	P	1	R	P	GR		
3	P	0	B	P	BB	Q	ϵ
4	P	0	G	P	BG		
5	P	1	B	P	BB		
6	P	1	G	P	GG	Q	ϵ
7	P	ϵ	R	Q	ϵ		
8	Q	0	B	Q	ϵ		
9	Q	1	G	Q	ϵ		
10	Q	ϵ	R	Q	ϵ		

$$d(p, a, Z) = [(q_1, h_1), (q_2, h_2), \ldots]$$

Figure 4.2. A two-state push-down automaton, recognizing the strings $\{ww^R \mid w \in (0,1)^*\}$, the palindromes in $\{0,1\}^*$.

$$\{ w w^R \mid w \in \{0,1\}^* \}$$

where w^R is the string w, but reversed in order.

This machine is nondeterministic because the moves in rows 3 and 6 contain alternative moves, and also because a move may be made on no input symbol (row 7), and alternatives to this exist.

The starting state is P, and the initial stack contents is R. It halts by empty stack.

Consider the input string 001100, and let us trace the machine configurations:

(P, 001100, R)	$\vdash$	(P, 01100, BR)	by row 1
or	$\vdash$	(Q, 001100, ϵ)	by row 7 (block)
(P, 01100, BR)	$\vdash$	(P, 1100, BBR)	by row 3a
or	$\vdash$	(Q, 01100, R)	by row 3b
(Q, 01100, R)	$\vdash$	(Q, 01100, ϵ)	by row 10 (block)
(P, 1100, BBR)	$\vdash$	(P, 100, GBBR)	by row 5
(P, 100, GBBR)	$\vdash$	(P, 00, GGBBR)	by row 6a
or	$\vdash$	(Q, 00, BBR)	by row 6b
(P, 00, GGBBR)	$\vdash$	(P, 0, BGGBBR)	by row 4
(P, 0, BGGBBR)	$\vdash$	(P, ϵ, BBGGBBR)	by row 3a (block)
or	$\vdash$	(Q, ϵ, GGBBR)	by row 3b (block)

(Q, 00, BBR)	$\vdash$ (Q, 0, BR)	by row 8
(Q, 0, BR)	$\vdash$ (Q, ϵ, R)	by row 8
(Q, ϵ, R)	$\vdash$ (Q, ϵ, ϵ)	by row 10 (accept)

The above trace shows the consequences of running down the various blind alleys that occur; nevertheless, the machine ends in an acceptance of the string.

This PDA does its work essentially by pushing B on the stack for every 0 and G for every 1 found in the first half of the input string. It must nondeterministically decide when the second half of the string begins. If it makes the wrong choice for a turning point, it will either exhaust its stack too soon, or it will exhaust its input string before the stack is empty.

It remains in state P for the first half of the string, and switches to state Q for the second half. While in state Q, the input string elements are effectively matched against the stack symbols; 0 must correspond to a B on the stack, and 1 to a G.

The initial stack symbol R becomes buried in the stack as soon as some symbols in w are matched, and becomes uncovered only as the last symbol in the input string is matched. Upon uncovering R, the stack is also emptied, permitting a halt. R is never pushed onto the stack; it therefore serves as an end marker for the stack.

If the input string is not a palindrome, the machine is unable to find a sequence of accepting moves. It will fail through an inability to match the stacked B's and G's against the 0's and 1's in the second half of the input list in state Q, or it will fail through exhausting the stack before exhausting the input list or vice versa.

Exercises

1. Trace the PDA of figure 4.2 for the strings 010010, 1010 and 0111. Show that the latter strings cannot be accepted by any sequence of moves.

2. Design a PDA that recognizes palindromes in the alphabet {0, 1, C}, where C marks the center of the palindrome; e.g., 0110C0110 is a legal palindrome. A deterministic PDA for this language exists.

3. Design a PDA that accepts strings in {(,)} consisting of all correctly nested parentheses, e.g., (( ) ( )) is in the language, but ( )) ( ) (is not.

4. Define a backtracking system for a PDA. The tape need not consist of fixed-length cells. Can you find a PDA for which the backtracking system will fail? Can you characterize the class of PDA's for which your backtracking system will fail?

Context-Free Languages and PDA

An important theorem connecting PDA and context-free languages is the following:

Theorem 4.1. For every context-free language L there exists a nondeterministic PDA M such that M exactly recognizes L, and conversely.

The practical importance of this theorem in compiler systems lies in the realization that essentially the syntax of every programming language in existence can be expressed almost exactly by a context-free grammar. Then theorem 4.1 insists that a mechanical recognizer for the language must exactly be a push-down automaton.

Of course, every regular grammar is also context-free, by its very form. The converse is not true. There are context-free languages that are not regular. The palindrome language is an example of a nonregular language, yet we have seen that a PDA can recognize it. (The proof that a palindrome cannot be regular is beyond the scope of this book). It may also be possible to transform a grammar which appears to be context-free into a regular grammar. However, in general such a transformation is not always possible, and when it is not, we must have a recognizer with a push-down stack.

The push-down stack in a compiler may be explicitly coded into the program, or it may be hidden. A recursive-descent compiler is an example of a top-down context-free recognizer in which the PDA push-down stack is in fact a stack of return addresses formed during the procedure calls which constitute the parsing process. Since this return address stacking system is invisible to the user of a modern programming language, the parsing process appears not to involve a stack.

Theorem 4.1 has two faces. The most interesting one from the point of view of compiler construction is that a given context-free language can be recognized by some PDA M. The converse, that given a PDA M, one can construct an equivalent grammar from it, is also true, but hardly needed in compiler construction. We shall therefore define the machine construction and prove equivalence to the language of the given grammar.

Algorithm 4.1. CFG to a Nondeterministic PDA

The machine M will be nondeterministic, will have only one state, q, and will have the rules defined as follows:

- $\delta(q, \epsilon, A)$ contains (q, w) for every production $A \rightarrow w$ in P.
- $\delta(q, a, a)$ contains (q, ϵ) for every a in Σ.

The stack symbols are in $N \cup \Sigma$, and the stack initially contains the start symbol S. Machine M will accept by empty stack.

Example. Consider the simple grammar

$$S \rightarrow 0S1 \mid c$$

This grammar describes the language $\{0\}c\{1\}$, where the number of 0's and 1's are equal. The transition function rules are:

$$\begin{aligned}
\delta(q, 0, 0) &= (q, \epsilon) \\
\delta(q, 1, 1) &= (q, \epsilon) \\
\delta(q, c, c) &= (q, \epsilon) \\
\delta(q, \epsilon, S) &= \{(q, 0S1), (q, c)\}
\end{aligned}$$

A trace of the machine moves for the string 00c11 is given next. We omit the state from the configurations, since it is always the same.

(00c11, S) $\vdash$	(00c11, 0S1)	(O.K.)
or	(00c11, c)	(N.G.)
(00c11, 0S1) $\vdash$	(0c11, S1)	
(0c11, S1) $\vdash$	(0c11, 0S11)	
or	(0c11, c1)	(N.G.)
(0c11, 0S11) $\vdash$	(c11, S11)	
(c11, S11) $\vdash$	(c11, 0S111)	(N.G.)
or	(c11, c11)	
(c11, c11) $\vdash$	(11, 11)	
(11, 11) $\vdash$	(1, 1)	
(1, 1) $\vdash$	(ϵ, ϵ)	(acceptance)

Another Example. Empty production rules are acceptable, too:

$$S \rightarrow 0S1 \mid \epsilon$$

The transition function is

$$\begin{aligned}
\delta(q, 0, 0) &= (q, \epsilon) \\
\delta(q, 1, 1) &= (q, \epsilon) \\
\delta(q, \epsilon, S) &= \{(q, 0S1), (q, \epsilon)\}
\end{aligned}$$

Then the string 0011 is accepted as follows:

$$(0011, S) \vdash (0011, 0S1) \vdash (011, S1) \vdash (011, 0S11)$$

$$\vdash (11, S11) \vdash (11, 11) \vdash (1, 1) \vdash (\epsilon, \epsilon), \text{ acceptance.}$$

Discussion

By following the example machine traces just given, it may be seen that the effect of the rule

$$\delta(q, a, a) = (q, \epsilon) \qquad \text{for all } a \in \Sigma$$

is to *match* input terminal symbols against terminal symbols on the stack top. The move is only possible when the two symbols match, and the result is to move the read head and pop the symbol from the stack. This matching operation ceases only when a nonterminal symbol appears on the stack top. Then the other rule:

$$\delta(q, \epsilon, A) \quad \text{contains } (q, w) \text{ for every } A \rightarrow w \text{ in } P$$

applies. The machine must somehow choose among one of several productions with the same left member; the left member is known, since it is on the stack top.

Once a choice is made, the nonterminal A is replaced on the stack by the string w, which in general contains both terminals and nonterminals.

Recall that the production rule choice problem was also present in attempting to construct a derivation tree for a sentence from the top down. We have not solved the choice problem, but are examining it from a different point of view.

Exercise

Construct a PDA for grammar G_0, given below, then show that it accepts the string (a+a)*a but not the string (+a).

$$\begin{aligned} E &\rightarrow E+T \mid T \\ T &\rightarrow T*F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

Proof of Theorem 4.1: We prove the stronger result expressed by the next lemma.

Lemma 4.1. $(q, wx, Ay) \vdash^* (q, x, y)$ in M if and only if $A \Rightarrow^* w$ in grammar G, where A is in N, wx is in Σ^*, and y is in $(N \cup \Sigma)^*$.

This lemma includes theorem 4.1 as a special case, with A = S, and $x = y = \epsilon$.

Proof: If Part. We prove the if part of the lemma by induction on the number of steps in the derivation $A \Rightarrow^* w$. Consider a derivation of one step. Then $A \rightarrow w$ is a production, with w a terminal string. This production implies that there is a transition

$$\delta(q, \epsilon, A) \text{ contains } (q, w)$$

which yields the machine move:

$$(q, wx, Ay) \vdash (q, wx, wy)$$

Now w is a terminal string. If its length $|w| = n$, then we may invoke n applications of the matching rules, which have the form:

$$\delta(q, a, a) = \{(q, \epsilon)\}$$

for every terminal "a" in the alphabet. These n applications yield the configuration

$$(q, x, y)$$

which was to be shown.

Now assume that the derivation $A \Rightarrow^* w$ requires k steps, where $k > 1$, and that lemma 4.1 holds for all $k' < k$. The first step of the derivation has the form

$$A \Rightarrow X_1X_2X_3 \ldots X_n \Rightarrow^* x_1x_2x_3 \ldots x_n = w,$$

where $X_1 \Rightarrow^* x_1$, $X_2 \Rightarrow^* x_2$, etc., and $X_1, \ldots, X_n$ are in $N \cup \Sigma$.

Thus the move

$$(q, wx, Ay) \vdash (q, x_1x_2 \ldots x_nx, X_1X_2 \ldots X_ny)$$

exists. Now if X_1 is a terminal symbol, it must be equal to x_1, and the transition $\delta(q, x_1, x_1) = \{(q, \epsilon)\}$ may be made. If X_1 is nonterminal, then by the inductive hypothesis (since $X_1 \Rightarrow^* x_1$ by less than k moves), the moves

$$(q, x_1r, X_1s) \vdash^* (q, r, s)$$

for any strings r and s exist.

Thus in either case, the following moves exist:

$$(q, x_1x_2 \ldots x_nx, X_1X_2 \ldots X_n\ y) \vdash^* (q, x_2 \ldots x_nx, X_2 \ldots X_ny)$$

More repetitions of this process, for $X_2, \ldots, X_n$ eventually yields the configuration (q, x, y), which was to be shown. QED

Proof: Only If part. Let $(q, wx, Ay) \vdash (q, x, y)$ in k moves; we prove that $A \Rightarrow^* w$ by induction on the number of moves, k.

For $k = 1$, w must be ϵ and $A \rightarrow \epsilon$ is in P. (There are no other possibilities with A on the top of the stack, in one machine move). Thus assume the lemma valid for all moves of length $k' < k$; we prove it valid for k. The first move must have the form

$$(q, wx, Ay) \vdash (q, wx, X_1 \ldots X_ny)$$

where $A \rightarrow X_1 \ldots X_n$ is in P, and

$$(q, x_i, X_i) \vdash^* (q, \epsilon, \epsilon)$$

for all $1 \leq i \leq n$ in k' moves or less, where $w = x_1x_2 \ldots x_n$.

Then $X_i \Rightarrow^* x_i$ for all i, by the inductive hypothesis; but putting all this together, we find

$$A \Rightarrow X_1 \ldots X_n \Rightarrow^* x_1 X_2 \ldots X_n \Rightarrow^* x_1 x_2 X_3 \ldots X_n \Rightarrow^* \ldots \Rightarrow^* x_1 x_2 \ldots x_n = w$$

QED

The Input List, the Stack, and Sentential Forms

An interesting property of the NDPDA defined in algorithm 4.1 is expressed by the following theorem:

Theorem 4.2. Let (q, y, h) be any configuration of the NDPDA for some grammar G, where the input string is xy, such that

$$(q, xy, S) \vdash^* (q, y, h)$$

Then xh is a left-most sentential form in G, i.e., $S \Rightarrow^* xh$.

The converse is also true: Given any sentential form xh in G, such that x is a terminal string, and at most one left-most symbol in h is terminal, then (q, y, h) is a configuration of the NDPDA such that

$$(q, xy, S) \vdash^* (q, y, h)$$

To visualize the significance of this theorem, consider figure 4.1(a). The string x is ". . .de", y is "abc..", and the stack string h is "ZWXY..". Then the theorem says that ". . .deZWXY.." is a sentential form. In short, the input list to the left of the read head, when concatenated with the stack, is a sentential form.

To prove this, note that it is trivially true for the initial configuration; the input string prefix x is empty, and S is on the stack. Therefore, assume that the theorem holds for n moves of the NDPDA, $n \geq 0$, and consider the $(n+1)$th move. This move either is a *matching* move, or a production rule *replacement* move. If a matching move, the string xh is clearly unchanged, since first symbol in h effectively is moved to the last position in x.

If a replacement move, the string xh before the move is of the form xAz, where $h = Az$. After the move, the string has the form xwz, where $A \rightarrow w$ is a production. By the inductive hypothesis, xAz is a sentential form, i.e.,

$$S \Rightarrow^* xAz$$

But since $A \rightarrow w$ is a production, xwz is also a sentential form:

$$S \Rightarrow^* xAz \Rightarrow xwz$$

QED

A proof of the converse is similar.

4.2. LL(k) Grammars

The LL(k) grammars are a proper subset of the context-free grammars. They are the largest such class that permits deterministic left-to-right top-down recognition with a lookahead of k symbols.

The deterministic top-down parsing problem is represented by figure 2.11, in chapter 2. We have some nonterminal node (T in the figure), and an uncompleted string, "(a*a)" in the figure. If the correct production can be deduced from the partially constructed tree and the next k symbols in the unscanned string, for every possible top-down parsing step, then we say that the grammar is LL(k). It is usually not obvious whether a given grammar is LL(k), nor can we ordinarily examine every possible parsing step. We seem to need two algorithms: one to test a grammar for the LL(k) condition and another one to generate a parsing table, similar to figure 2.16, for some grammar. It turns out that only the latter algorithm is needed; the failure to generate a parsing table will be apparent from the algorithm, and this failure will mean that the grammar is not LL(k). If the table generation succeeds, then the grammar is LL(k), and we will not only have a definition of a deterministic parser, but will also know that the grammar is unambiguous.

4.2.1. Definitions

The discussion on lookahead of k symbols is considerably simplified if there always exist at least k symbols to examine. For this reason, we introduce a new terminal symbol $\perp$ not already in the grammar, and append k of these symbols to every input string, prior to parsing. Hence we introduce a new nonterminal S′ and a production

$$S' \rightarrow S \perp^k$$

where S is the grammar's start symbol, and $\perp^k$ is a string of k symbols $\perp$. The symbol S′ becomes the new start symbol.

Definition 1. A production $A \rightarrow x_1$ in a context-free grammar G is called an *LL(k) production*, if in G,

$$S' \Rightarrow^* w\, A\, y \Rightarrow w\, x_1\, y \Rightarrow^* w\, z\ldots, \text{ and}$$

$$S' \Rightarrow^* w\, A\, y' \Rightarrow w\, x_2\, y' \Rightarrow^* w\, z\ldots$$

with $|z| = k$ implies $x_1 = x_2$.

Definition 2. A nonterminal in a context-free grammar is called an *LL(k) nonterminal* if all its productions are LL(k) productions.

Example. In the grammar

$$S \rightarrow Abc \mid aAcb$$

$$A \rightarrow \epsilon \mid b \mid c$$

S is an LL(1) nonterminal, while A is an LL(2) nonterminal. The strings derivable from S are (first production) bc, bbc, cbc, and (second production) acb, abcb, accb. Clearly, the second production is uniquely selected on the lookahead symbol "a", while the first is selected on symbols "b" or "c".

For the A productions, we need to consider the sentential forms in which A is embedded, e.g., "Abc" and "aAcb". By the definition, the string preceding A is different, so we need only consider whether the three A productions can be distinguished within "Abc" and within "aAcb". In the former, we have the three lookahead strings "bc", "bb" and "cb"; these are distinguishable with k = 2 but not with k = 1. In the latter production, the lookahead strings are "cb", "bc", and "cc", again distinguishable with k = 2 but not k = 1. The A productions are therefore LL(2), but not LL(1).

Note that if the lookahead sets were considered without regard to the particular sentential form prefix w in $S \Rightarrow^* wAy$, then the A nonterminal requires three lookahead symbols.

It is clear that an LL(k) grammar is also an LL(k−1) grammar, for $k \geq 1$.

4.2.2. Some Properties

Theorem 4.2.1. Each LL(k) grammar is unambiguous.

An ambiguity would lead to a contradiction with definitions 1 and 2.

Theorem 4.2.2. An LL(k) grammar has no left-recursive nonterminals, i.e., nonterminals A such that $A \Rightarrow^+ Aw$ for some w.

Proof: Suppose that a nonterminal A_0 is left-recursive. Then there is some sequence of nonterminals $A_0, A_1, \ldots, A_n$ that are all left-recursive, and such that

$$A_0 \Rightarrow^+ A_1x_1 \Rightarrow^+ A_2x_2 \Rightarrow^+ \ldots \Rightarrow^+ A_nx_n$$

and $A_n = A_0$. Now at least one of these must have another production, otherwise all of them would be useless and could be deleted from the grammar. Also, at least one of the x_i must be incapable of deriving the empty string; if they all could, then we could have a derivation sequence $A_0 \Rightarrow^+ A_0$,

and an obvious ambiguity. Thus one of the A_i (call it A from here on) is such that

$$A \Rightarrow Bx' \Rightarrow^+ Ax$$

where $|x| > 0$, and also $A \rightarrow y$, where y is not Bx'. Then we can construct the derivation sequences

$$S \Rightarrow^+ rAx^k \ldots \Rightarrow rBx'x^k \ldots \Rightarrow rAx^{k+1} \ldots \Rightarrow ryx^{k+1} \ldots \Rightarrow^* rz \ldots$$

$$S \Rightarrow^+ rAx^k \ldots \Rightarrow ryx^k \ldots \Rightarrow^* rz \ldots$$

where $|z| = k$, and they contradict the assumption that the grammar is LL(k). QED

The LL(k) definition states that, given a left-most sentential form wAx, such that w matches the first |w| symbols of the input string, then the next production $A \rightarrow y$ can be inferred from the next k input symbols. It appears that in order to select the production $A \rightarrow y$ we need a mapping of all strings wAx' to the production set, where $|x'| = k$. The resulting LL(k) parsing table would be impossibly large. The following two theorems show that we do not need such a complete mapping—a mapping of strings Ax' to the production set is sufficient. Before we can introduce these theorems, we need to develop two useful functions $FIRST_k(w)$ and $FOLLOW_k(w)$.

FIRST and FOLLOW sets

The domain of $FIRST_k$ is some string w in $(N \cup \Sigma)^*$ and the domain of $FOLLOW_k$ is a nonterminal A in N. The functions are then defined as follows:

$$FIRST_k(w) = \{ x \mid w \Rightarrow^* xy, xy \in \Sigma^* , \text{ and}$$

$$(\text{if } |xy| < k \text{ then } y = \epsilon \text{ else } |x| = k)\}$$

$$FOLLOW_k(A) = \{ x \mid S \Rightarrow^* uAy \text{ and } x \in FIRST_k(y)\}$$

where the derivations are left-most. That is, $FIRST_k(w)$ for some string w is the set of all leading terminal strings of length k or less in the strings derivable from w. A string x in $FIRST_k(w)$ is less than k in length only if x is fully derivable from w, i.e., $w \Rightarrow^* x$, and $|x| < k$. The empty string is in $FIRST_k(w)$ if $w \Rightarrow^* \epsilon$. Note that w may include or consist of nonterminals.

$FOLLOW_k(A)$ is the set of all derivable terminal strings of length k or less that can follow A in some left-most sentential form. The empty string ϵ will be in $FOLLOW_k(A)$ if A is the last symbol in some sentential form. In particular, ϵ is in $FOLLOW_k(S)$, where S is the start symbol.

With these definitions, the following useful properties can readily be proven. In these, the range of the $FIRST_k$ set is extended to include a set of strings; i.e., $FIRST_k(U\ V)$, where U and V are sets of strings, is the union of all $FIRST_k(u\ v)$, where u is in U and v is in V. Also $FIRST_k(w) = \epsilon$ for $k \leq 0$.

1. $FIRST_k(aw) = a\ FIRST_{k-1}(w)$ for any string w, where a is in Σ.
2. $FIRST_k(\epsilon) = \{\epsilon\}$.
3. $FIRST_k(x\ y) = \overline{FIRST}_k(\overline{FIRST}_k(x)\ \overline{FIRST}_k(y)) = FIRST_k(x\ FIRST_k(y)) = FIRST_k(FIRST_k(x)\ y)$.
4. Given a production $A \rightarrow w$ in G, $FIRST_k(A)$ contains $FIRST_k(w)$.
5. Given a production $A \rightarrow xXy$ in G, $FOLLOW_k(X)$ contains $FIRST_k(y\ FOLLOW_k(A))$. Note that y may be the empty string.
6. $FOLLOW_k(S)$ contains ϵ, where S is the start symbol of G.

Properties 1 and 2 should be obvious from the definitions. Property 3 expresses associativity of the $FIRST_k$ operator and concatenation; however, note that $FIRST_k(x\ y)$ is not identical to $FIRST_k(x)\ FIRST_k(y)$, since the former may be of length k at most and the latter may be of length greater than k.

Property 4 follows immediately from the definition.

Example. Consider the following simple grammar G_1. Let us compute $FIRST_k$ and $FOLLOW_k$ for its nonterminals, for k = 1:

1. $G \rightarrow E\ \perp$
2. $E \rightarrow TE'$
3. $E' \rightarrow +E$
4. $E' \rightarrow \epsilon$
5. $T \rightarrow FT'$
6. $T' \rightarrow *T$
7. $T' \rightarrow \epsilon$
8. $F \rightarrow (E)$
9. $F \rightarrow a$

- $FIRST_1(F) = \{ (, a\}$ from productions 8 and 9
- $FIRST_1(T') = \{*, \epsilon\}$ from productions 6 and 7
- $FIRST_1(T) = FIRST_1(FT') = FIRST_1(F\ FIRST_1(T')) = \{ (, a\}$
- $FIRST_1(E') = \{ +, \epsilon\}$ from productions 3 and 4
- $FIRST_1(E) = FIRST_1(TE') = \{ (, a\}$
- $FIRST_1(G) = FIRST_1(E) = \{ (, a\}$

- $\text{FOLLOW}_1(E) = \{\perp,)\} \cup \text{FOLLOW}_1(E')$ from productions 1, 3, and 8
- $\text{FOLLOW}_1(G) = \{\epsilon\}$ using property (6)
- $\text{FOLLOW}_1(E') = \text{FOLLOW}_1(E)$ from production 2
- $\text{FOLLOW}_1(T) = \text{FIRST}_1(E' \text{FOLLOW}_1(E)) \cup \text{FOLLOW}_1(T')$ from productions 2 and 6
- $\text{FOLLOW}_1(T') = \text{FOLLOW}_1(T)$ from production 5
- $\text{FOLLOW}_1(F) = \text{FIRST}_1(T' \text{FOLLOW}_1(T))$ from production 5

By using these relations repeatedly, we obtain the following table of FOLLOW_1 sets:

	1	2	3	4	5	6
G	ϵ					
E	$\perp$	)				
E′			$\perp$	)		
T			$\perp$	)	+	
T′			$\perp$	)	+	
F			$\perp$	)	+	*

Columns 1 and 2 are the contents of $\text{FOLLOW}_1(G)$ and $\text{FOLLOW}_1(E)$ known directly from the productions. The remaining columns are determined by inference from these and the FOLLOW_1 and FIRST_1 relations given above.

Exercises

1. A grammar and its FIRST_1 and FOLLOW_1 sets are given below. Verify the grammar's FIRST and FOLLOW sets:

   $S \rightarrow SA \mid Ba$
   $A \rightarrow Ab \mid B \mid \epsilon$
   $B \rightarrow aA \mid c$

nonterminal	FIRST_1	FOLLOW_1
S	a, c	ϵ, a, b, c
A	ϵ, a, b, c	ϵ, a, b
B	a, c	ϵ, a, b

2. Prove the six properties from the definitions.

3. Develop an algorithm based on the six properties that yields the FIRST_k and FOLLOW_k sets for any grammar.

Some Implications of the LL(k) Definition

We now use the FIRST and FOLLOW functions in a set of theorems that will lead to an efficient LL(1) parser.

Theorem 4.2.3. Let G be a context-free grammar. Then G is LL(k) if and only if: For every distinct pair of productions $A \rightarrow u$ and $A \rightarrow v$, and every left-most sentential form wAx derivable in G, $\text{FIRST}_k(ux) \cap \text{FIRST}_k(vx) = \varnothing$.

The proof is left to an exercise.

Now suppose that G has no empty productions and that G is LL(1). Then neither u nor v are empty, nor can either of these derive ϵ. Clearly x doesn't matter, since $\text{FIRST}_1(ux) = \text{FIRST}_1(u)$ and $\text{FIRST}_1(vx) = \text{FIRST}_1(v)$. It also means that the particular sentential form in which A appears is no longer important. We then have the weaker condition expressed in the following theorem.

Theorem 4.2.4. Let G be a context-free grammar with no empty productions. Then G is LL(1) if and only if, for every pair of productions $A \rightarrow u$ and $A \rightarrow v$, $\text{FIRST}_1(u) \cap \text{FIRST}_1(v) = \varnothing$.

The proof is trivial, given Theorem 4.2.3.

Is there a similar LL(1) condition for grammars with empty productions? Consider Theorem 4.2.3 again. If G contains empty productions, then u or v can be empty, or can derive empty strings. If $u \rightarrow^* \epsilon$ is possible, then clearly $\text{FIRST}_1(ux)$ includes $\text{FIRST}_1(x)$, and $\text{FIRST}_1(x)$ need not be a subset of $\text{FIRST}_1(u)$. However, note that x follows A in the left-most sentential form wAx of the theorem, and this suggests the following theorem:

Theorem 4.2.5. Let G be a context-free grammar. Then G is LL(1) if and only if, for every pair of productions $A \rightarrow u$ and $A \rightarrow v$ the following condition holds:

$$\text{FIRST}_1(u\ \text{FOLLOW}_1(A)) \cap \text{FIRST}_1(v\ \text{FOLLOW}_1(A)) = \varnothing$$

The FIRST-FOLLOW condition is certainly necessary for G to be LL(1). If u can derive ϵ, then $\text{FIRST}_1(ux)$ contains $\text{FIRST}_1(x)$ and $\text{FIRST}_1(x)$ is a subset of $\text{FOLLOW}_1(A)$. We must show that the condition is also sufficient—which is perhaps surprising, since $\text{FOLLOW}_1(A)$ is not related to any one sentential form, but rather the entire class of sentential forms containing A. We therefore prove sufficiency—the "if" part of the theorem.

Proof: Suppose that G were not LL(1). Then there exists a pair of derivations

$$S' \Rightarrow^* wAx \Rightarrow wux \Rightarrow^* wz$$

$$\text{and } S' \Rightarrow^* wAx \Rightarrow wvx \Rightarrow^* wz'$$

where $u \neq v$, and $\text{FIRST}_1(z) = \text{FIRST}_1(z')$. Now

$$ux \Rightarrow^* z$$

$$\text{and } vx \Rightarrow^* z'$$

Now let $z = u'x'$ and $z' = v'x''$ where $u \Rightarrow^* u'$, $x \Rightarrow^* x'$, $v \Rightarrow^* v'$, and $x \Rightarrow^* x''$. Note that $\text{FIRST}_1(x')$ is in $\text{FOLLOW}_1(A)$, and $\text{FIRST}_1(x'')$ is also in $\text{FOLLOW}_1(A)$. We now examine three cases, depending on whether u' or v' or neither or both, are empty:

CASE 1

$u' \neq \epsilon$ and $v' \neq \epsilon$. Then clearly

$$\text{FIRST}_1(z) \subset \text{FIRST}_1(u) \subset \text{FIRST}_1(u\ \text{FOLLOW}(A))$$

$$\text{and } \text{FIRST}_1(z') \subset \text{FIRST}_1(v) \subset \text{FIRST}_1(v\ \text{FOLLOW}(A))$$

Then

$$\text{FIRST}_1(u) \cap \text{FIRST}_1(v) \neq \varnothing$$

QED
(We have proven that premise P implies Q by proving that $\sim$Q implies $\sim$P).

CASE 2

$u' = \epsilon$ and $v' \neq \epsilon$. Here,

$$\text{FIRST}_1(z') \subset \text{FIRST}_1(v) \cap \text{FIRST}_1(v\ \text{FOLLOW}(A))$$

$$\text{and } \text{FIRST}_1(z) = \text{FIRST}_1(x') \subset \text{FOLLOW}_1(A) \subset \text{FIRST}_1(u\ \text{FOLLOW}(A))$$

Again,

$$\text{FIRST}_1(u\ \text{FOLLOW}(A)) \subset \text{FIRST}_1(v\ \text{FOLLOW}(A)) \neq \varnothing$$

CASE 3

$u' = \epsilon$, $v' = \epsilon$. Here,

$$\text{FIRST}_1(z) = \text{FIRST}_1(x') \subset \text{FOLLOW}_1(A) \subset \text{FIRST}_1(u\ \text{FOLLOW}(A))$$

$$\text{and } \text{FIRST}_1(z') = \text{FIRST}_1(x'') \subset \text{FOLLOW}_1(A) \subset \text{FIRST}_1(v\ \text{FOLLOW}(A))$$

QED

Theorem 4.2.5 suggests the following generalization:

G is LL(k) if and only if for every pair of productions $A \rightarrow u$ and $A \rightarrow v$, $FIRST_k(u\ FOLLOW_k(A)) \cap FIRST_k(v\ FOLLOW_k(A)) = \varnothing$.

Unfortunately, this statement is not true for $k>1$. A grammar satisfying the above condition is said to be *strong LL(k)*, and it happens that not all strong LL(k) grammars are LL(k), as the example given earlier shows:

Let G have the productions

$$S \rightarrow Abc \mid aAcb$$
$$A \rightarrow \epsilon \mid b \mid c$$

Now $FOLLOW_2(A) = \{bc, cb\}$. Then for $A \rightarrow \epsilon$ we have

$$FIRST_2(\epsilon\ FOLLOW_2(A)) = \{bc, cb\}$$

For $A \rightarrow b$ we have $FIRST_2(b\ FOLLOW_2(A)) = \{bb, bc\}$. Hence this is not strong LL(2), yet it is LL(2), as we have shown previously.

Essentially, the set $FIRST_k(u\ FOLLOW_k(A))$ for a production $A \rightarrow u$ contains all of the k-symbol lookaheads derivable from Ax in all the left-most sentential forms wAx. However, a top-down decision step in an LL(k) parser is based on a particular left-most sentential form wAx, and the possible k-symbol lookaheads derivable from Ax is a subset of $FIRST_k(u\ FOLLOW_k(A))$, for a production $A \rightarrow u$.

The case $k>1$ is different from the case $k = 1$ essentially because a pair of symbols comprising a lookahead string can stem from a combination of derivations based on A and on the strings following A, and the combination causes certain grammars to be strong LL(k) but not LL(k).

4.3. Deterministic LL(1) Parser

The push-down automaton given in section 4.1 is nondeterministic for only one reason—when a nonterminal symbol A appears on the top of the stack, then a choice must be made among the set of productions $A \rightarrow w_1 \mid w_2 \mid w_3 \mid \ldots$ in P.

We therefore see that to make this parser deterministic, we need a table to select one of several productions. Such a table will consider the stack top symbol A and the next k input symbols in the input list; based on this pair, it must uniquely select a production $A \rightarrow w_i$.

An *LL(k) selector table* maps a pair (X,u) to a production $X \rightarrow w$, where X is a nonterminal symbol (on the stack top), and u is some terminal string, of length k. Such a table must yield a many-to-one mapping, since many possible strings u may correspond to a given production, yet a production

must be uniquely selected from the pair (X,u). The table size is finite, since there are a finite number of nonterminals and terminal strings of fixed length k.

Theorem 4.2.5 provides the justification for such a table, and also states that the grammars that can be so parsed are the strong LL(k) grammars.

4.3.1. LL(1) Selector Table

A finite selector table can be constructed for any k. However, its size grows very rapidly with k for a typical grammar. It turns out that in practice, if a grammar is not LL(1), it will likely not be LL(k) for any k. It is better to transform the grammar in an attempt to find an LL(1) grammar, rather than attempt to construct a parser with $k > 1$. We therefore restrict the following discussion to $k = 1$.

Note that a LL(0) grammar can contain at most one production $A \rightarrow w$ for each nonterminal A, and is therefore useless for any practical purpose–at most one terminal string can be derived in the grammar.

Before we describe the construction of a selector table, let us examine a typical LL(1) selector table, figure 4.3. This table describes a grammar G_1 whose language is the class of arithmetic expressions with all four operations $+ \; - \; * \; /$ and parenthesizing. The productions for the grammar are given in the right-hand column of figure 4.3. The language of this grammar is essentially the arithmetic language $L(G_0)$, chapter 2, however, G_0 is not LL(k) for any k, since it contains left-recursive productions, while G_1 is LL(1). Grammar G_0 can be transformed into G_1 by means to be described later.

LL(1) Parser Algorithm

1. Initially, the parser PDA contains the start symbol S on its stack top.
2. If the stack top contains a terminal symbol "a", then the input symbol must be an "a", else ERROR. If the two match, then advance the read head and pop the terminal symbol from the stack.
3. If the stack top contains a nonterminal symbol A, then examine the input symbol currently under the read head, and consult the LL(1) selector table (figure 4.3, for example) to determine which production is to be applied. If the selector table indicates production $A \rightarrow w$, then remove A from the stack and push the string w onto the stack.
4. The machine halts by empty stack.

Let us try this machine on the string "a – a – a". The initial configuration is therefore:

Non-terminal		Input	Production
1	E	a, (	$E \rightarrow TE''$
2	E″	+, -	$E'' \rightarrow T'E''$
3	E″	), ϵ	$E'' \rightarrow \epsilon$
4	T′	+	$T' \rightarrow +T$
5	T′	-	$T' \rightarrow -T$
6	T	a, (	$T \rightarrow FT''$
7	T″	*, /	$T'' \rightarrow F'T''$
8	T″	+, -,), ϵ	$T'' \rightarrow \epsilon$
9	F′	*	$F' \rightarrow *F$
10	F′	/	$F' \rightarrow /F$
11	F	a	$F \rightarrow a$
12	F	(	$F \rightarrow (E)$

Figure 4.3. An LL(1) selector table, for grammar G_1.

$$[a-a-a, E]$$

since E is the start symbol of the grammar. The selector table row 1 indicates that with E on the stack top and "a" the next symbol, $E \rightarrow TE''$ should be applied; TE'' replaces E on the stack top, yielding the configuration

$$[a-a-a, TE'']$$

Next, the selector table indicates that T should be replaced by FT'' on the stack top, yielding

$$[a-a-a, FT''E'']$$

Then the selector table indicates replacing F by a, yielding

$$[a-a-a, aT''E'']$$

At this point, the matching rule (2) applies, yielding the configuration

$$[-a-a, T''E'']$$

The remaining moves are

$$[-a-a, T''E''] \vdash [-a-a, E'']$$

$$\vdash [-a-a, T'E''] \vdash [-a-a, -TE'']$$

$$\vdash [a-a, TE''] \vdash [a-a, FT''E'']$$

$$\vdash [a-a, aT''E''] \vdash [-a, T''E'']$$

$$\vdash [-a, E''] \vdash [-a, T'E''] \vdash [-a, -TE''] \vdash [a, TE'']$$

$$\vdash [a, FT''E''] \vdash [a, aT''E''] \vdash [\epsilon, T''E''] \vdash [\epsilon, E'']$$

$$\vdash [\epsilon, \epsilon]$$

The derivation tree for this sentence and grammar is shown in figure 4.4. A comparison of the machine moves with this tree reveals that the machine effectively constructs the tree top-down by a left-to-right tree scan. That is, the derivation is

$$E \Rightarrow TE'' \Rightarrow FT''E'' \Rightarrow aT''E'' \ldots$$

for the first few steps.

Selector Table Construction

Most of our work is done. Given an algorithm for the FIRST and FOLLOW sets, an LL(1) selector table is very easy to build:

Let $A \rightarrow w$ be any production in the grammar, and let the terminal symbol x be in the set $FIRST_1(w\ FOLLOW_1(A))$. Then the selector table pair (A, x) maps to production $A \rightarrow w$.

If any pair (A,x) maps to two or more different productions, then the grammar cannot be LL(1); we say that we have a conflict.

For example, consider the grammar G_1 given previously. The selector table for this grammar is given next and is seen to be free of conflicts:

Pair	Production
E, (	$E \rightarrow TE'$
E, a	$E \rightarrow TE'$
E′, +	$E' \rightarrow +E$
E′,)	$E' \rightarrow \epsilon$
E′, ⊥	$E' \rightarrow \epsilon$
T, (	$T \rightarrow FT'$
T, a	$T \rightarrow FT'$
T′, *	$T' \rightarrow *T$
T′, +	$T' \rightarrow \epsilon$
T′, ⊥	$T' \rightarrow \epsilon$
T′,)	$T' \rightarrow \epsilon$
F, (	$F \rightarrow (E)$
F, a	$F \rightarrow a$
G, (	$G \rightarrow E\perp$
G, a	$G \rightarrow E\perp$

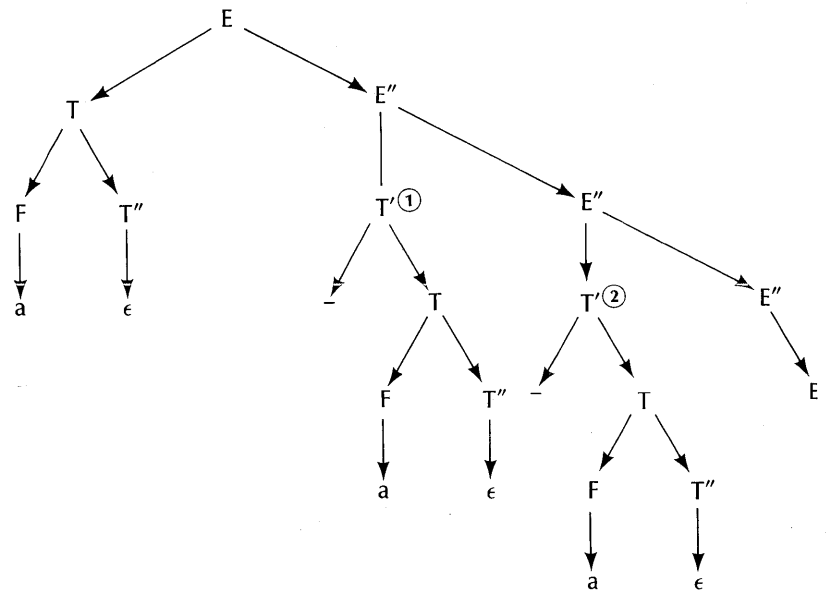

Figure 4.4. Derivation tree for string "a – a – a" in grammar G_1.

On the other hand, the following simple grammar has a selector table containing conflicts; it is therefore not LL(1):

$$S \rightarrow A\perp$$
$$A \rightarrow B \mid C$$
$$B \rightarrow aC$$
$$C \rightarrow aa$$

Pair	Production
S, a	$S \rightarrow A\perp$
A, a	$A \rightarrow B$
A, a	$A \rightarrow C$
B, a	$B \rightarrow aC$
C, a	$C \rightarrow aa$

The difficulty is with the pair of productions $A \rightarrow B$ and $A \rightarrow C$, both of which correspond to the selector pair (A, a). The parser cannot deterministically map (A, a) to a single production, hence is not LL(1). Both B and C derive strings beginning with "a", hence the PDA cannot decide between the productions $A \rightarrow B$ and $A \rightarrow C$ when "a" is the next input symbol.

4.3.2. LL(1) Grammar Transformations

We now consider a transformation that is often effective on a grammar containing a left-recursion, which (1) will preserve the semantic ordering of binary operations, and (2) will usually succeed in generating an LL(1) grammar.

Given a simple left-recursion of the form

$$A \rightarrow Ax \mid Ay \mid \ldots \mid w \mid z \mid \ldots$$

we first *stratify* this production set by introducing two new nonterminals B and C, as follows:

$$\begin{aligned} A &\rightarrow AB \mid C \\ B &\rightarrow x \mid y \mid \ldots \\ C &\rightarrow w \mid z \mid \ldots \end{aligned}$$

Note that B collects the strings past the A in the left-recursive A productions, and that C collects all the other A production right-hand parts. Then the productions $A \rightarrow AB \mid C$ are rewritten:

$$\begin{aligned} A &\rightarrow CA' \\ A' &\rightarrow BA' \mid \epsilon \end{aligned}$$

where A′ is another new nonterminal.

Here is an example, which yields the production set G_1 displayed in the selector table in figure 4.3, as based on grammar G_0:

The basis grammar is:

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow a \mid (E) \end{aligned}$$

which is an obvious extension of G_0.

We first stratify the E productions, introducing the new nonterminals E″ and T′:

Write $T' \rightarrow + T \mid - T$, so that the E productions become:

$$E \rightarrow E\ T' \mid T$$

Then these two productions are rewritten as:

$$\begin{aligned} E &\rightarrow T\ E'' \\ E'' &\rightarrow T'\ E'' \mid \epsilon \end{aligned}$$

In a similar way, the T productions are rewritten:

$$F' \rightarrow * \ F \mid / \ F$$
$$T \rightarrow F \ T''$$
$$T'' \rightarrow F' \ T'' \mid \epsilon$$

The F productions remain unchanged. We clearly obtain the production set displayed in figure 4.3. The reader should verify that the resulting grammar is LL(1) by verifying the selection table given in the figure. Figure 4.4 displays a derivation tree for the sentence "a−a−a", which shows that the two operators are correctly associated.

Exercises

1. Design a table-driven LL(1) implementation, using the sparse table approach described in chapter 3. Write a table interpreter in some language of your choice.

2. Add unary minus, logical operators and indexed variables to grammar G_1, and attempt to transform it into an LL(1) grammar.

3. Construct a selector table for the grammar

$$S \rightarrow 0S0 \mid 1S1 \mid \epsilon$$

and trace the PDA for the strings 0110 and 0011.

4. Consider the LL(1) parser and grammar given in figure 4.3. Let the parser emit "LOAD a" on production $F \rightarrow a$, ADD on production $T' \rightarrow +T$, SUB on production $T' \rightarrow -T$, MPY on production $F' \rightarrow *F$ and DIV on production $F' \rightarrow /F$. Show through selected examples that the system emits valid reverse Polish code for arithmetic instructions, correctly associated and distributed.

4.4. Recursive Descent Parsers

The recursive descent parsers are closely related to the LL(1) parsers. They are among the most popular of the compiler parsers, perhaps because the parsing and semantics operations appear together as a reasonably lucid and self-explanatory program. The method uses procedure calls and other programming techniques familiar to most programmers. It is a top-down method, and we shall show that a necessary condition for a recursive descent

compiler to operate correctly is that its source grammar be LL(1). We shall explore two recursive descent synthesis methods. The first is an algorithm that accepts an LL(1) grammar and generates a recursive descent compiler program, in the form of a sequence of statements in an Algol-like language. The program produced by this method can be proven correct without much effort and illustrates the connection of recursive descent to LL(1) grammars; however, it is rather inefficient, as it exploits only a few of the control structures available in a high-level language.

The second recursive descent method accepts an extended BNF grammar. Such a grammar may contain closure, alternation, and concatenation operators, similar to those in regular expressions. With these features, a more satisfying recursive descent parser may be constructed. It, too, is related to the LL(1) grammars; however, the LL(1) concepts must also be extended to the new grammar form, and we shall show how this is done, and how the parser may be certified valid relative to its basis grammar.

The basic notion of recursive descent is that each nonterminal in a grammar is associated with a recursive procedure. The task of the procedure is to accept any string derivable from that nonterminal, and only such strings. It must furthermore move the read head through the string only if it accepts it, and not move the read head if it fails to accept it. Within such a procedure, the alternative right parts of the productions associated with the procedure are tested one at a time. It is essential that for any string in which one alternative succeeds, none of the others may succeed.

It can be seen from this brief introduction that a recursive descent parser is a top-down system, and that the central problem of validation for the parser is that the alternatives within a production set with a common left member be distinguishable through their first symbol. We shall explore this notion in greater detail upon formally defining a recursive descent parser generator.

Example of a Recursive Descent Parser. Figure 4.5 displays a recursive descent parser for an arithmetic grammar containing the four arithmetic operations. It consists of three procedures, EXPR, TERM, and FACT, and a Boolean variable FLAG. These procedures make use of a variable CHARACTER, which contains the next character in the input list, and a procedure NEXTCHAR, which fetches the next character. The input identifiers are assumed to be letters.

The procedures EXPR, TERM, and FACT must be recursive; that is, their return addresses must be stacked. Since they have no local variables or call parameters, nothing else need be stacked.

The task of procedure EXPR is to examine the input string beginning with the current symbol CHARACTER, and attempt to scan an arithmetic expression, where an *expression* consists of a term or a list of terms separated by the operators + and −. The task of procedure TERM is to identify a term, where a *term* is a factor or a list of factors separated by the operators *

```
 1  procedure EXPR; {recognizes expressions in add/subtract operators }
 2  begin
 3    TERM;
 4    if not FLAG then return;
 5    while FLAG do
 6    begin
 7      if CHARACTER = '+' then
 8      begin
 9        NEXTCHAR;
10        TERM
11      end
12     else
13      if CHARACTER = '-' then
14      begin
15        NEXT CHAR;
16        TERM
17      end
18     else FLAG: = FALSE
19    end;
20    FLAG: = TRUE;
21    return
22  end; (procedure EXPR)

24  procedure TERM; {recognizes expressions in * and /}
25  begin
26    FACT;
27    if not FLAG then return;
28    while FLAG do
29    begin
30      if CHARACTER = '*' then
31      begin
32        NEXTCHAR;
33        FACT
34      end
35     else
36      if CHARACTER = '/' then
37      begin
38        NEXTCHAR;
39        FACT
40      end
41     else FLAG: = FALSE
```

Figure 4.5. A recursive descent compiler for a simple arithmetic grammar, essentially G_0 extended with all four arithmetic operations.

```
42    end;
43    FLAG: = TRUE;
44    return
45  end; {procedure TERM}

47  procedure FACT; { recognizes operands and parenthesized
    expressions }
48  begin
49    if CHARACTER > = 'A' and CHARACTER < = 'Z' then
50    begin
51      NEXTCHAR;
52      FLAG: = TRUE
53    end
54   else
55    if CHARACTER = '(' then
56    begin
57      NEXTCHAR:
58      EXPR;
59      if not FLAG then return;
60      if CHARACTER = ')' then
61      begin
62        NEXTCHAR;
63        FLAG: = TRUE
64      end
65     else FLAG: = FALSE;
66    end
67   else FLAG: = FALSE;
68    return
69  end {procedure FACT}
```

Figure 4.5. (cont'd.)

and /. The task of FACT is to identify a factor, where a *factor* is a parenthesized expression or an identifier. Each of these moves the read head as far as possible, considering the form it is asked to identify, possibly making no move at all. The variable FLAG is set to TRUE if a match was possible, such that the read head moved, and set to FALSE if a match was not possible. The state of the FLAG upon emerging from a call on any of these procedures then indicates whether another alternative should be tested, or a syntax error has occurred, or whether the recognition was successful to this point.

An input string is parsed by setting CHARACTER to its first character, then calling EXPR. Procedure EXPR calls the other procedures and these may end up calling EXPR again recursively. Eventually, EXPR terminates

with FLAG set to TRUE if the string is an expression and FALSE otherwise. A termination with FLAG = FALSE is such that CHARACTER is the first character in error in the input string.

It is not obvious that these procedures operate correctly. We shall discuss this point later. For now, consider the string A*(B − C). A trace of the calls and operations within the procedures for this string is given in figure 4.6.

In the trace, EXPR first calls TERM, which first calls FACT. FACT then examines whether the first character is "(" or LETTER. An expression may only begin with one of them. If one or the other is seen, the next character is fetched by a CALL NEXTCHAR, which also deposits it in CHARACTER, and FLAG is set TRUE. Program control then returns to line 27 in procedure TERM.

TERM next sees if a "*" or a "/" is in CHARACTER. These symbols are legal after LETTER, but are not the only legal operators; so are "+" and "−". It happens that "*" is seen next. Thus TERM scans "*", advances past it (line 34), and calls FACT again.

The read head is now positioned at CHARACTER = "(". On "(", FACT calls EXPR and then looks for a ")". A failure to find EXPR, then ")" is a syntax error.

EXPR is therefore launched on the string "B − C)". It will scan the "B − C" portion, but will return on the ")". Note that it reports TRUE to the calling program FACT, line 58, so that the ")" will next be scanned.

Eventually, the input string is exhausted. It first happens in FACT, when the ")" is scanned. Technically, CHARACTER should contain $\perp$ when the end of the input string is reached, so that the remaining tasks will yield valid results. The result is an "unwinding" of the recursive calls, with the end result a return from EXPR with FLAG = TRUE.

Exercises

1. Trace the following strings through the program of figure 4.5, and indicate whether they are valid or invalid.

   (A)
   A+B*C
   A++B
   −C

2. Why is FLAG set to TRUE near the end of the EXPR routine?

3. What is the purpose of the WHILE loop in EXPR and TERM?

Remaining String	Procedure	FLAG	Line	Comment
A*(B-C)	EXPR	- - -	2	Enter procedure EXPR
	EXPR	- - -	3	Call TERM
	. TERM	- - -	26	Call FACT
	. . FACT	- - -	49	CHARACTER is LETTER
*(B-C)	. . FACT	TRUE	52	Input advanced, FLAG set TRUE
	. . FACT	TRUE	68	Return from FACT to TERM
	. TERM	TRUE	27	Return point in TERM
	. TERM	TRUE	28	Start WHILE-DO loop
	. TERM	TRUE	30	CHARACTER is "*"
(B-C)	. TERM	TRUE	33	Call FACT
	. . FACT	TRUE	49	CHARACTER is not LETTER
	. . FACT	TRUE	55	CHARACTER is "("
B-C)	. . FACT	TRUE	58	Call EXPR
	. . . EXPR	TRUE	3	Into EXPR, call TERM
	. . . . TERM	TRUE	26	Into TERM, call FACT
-C)	. . . . . FACT	TRUE	52	CHARACTER is LETTER
	. . . . TERM	TRUE	28	Start WHILE-DO
	. . . . TERM	FALSE	41	CHARACTER is neither "*" nor "/"
	. . . . TERM	TRUE	43	But that's OK
	. . . EXPR	TRUE	5	Start WHILE-DO
	. . . EXPR	TRUE	7	CHARACTER is not "+"
	. . . EXPR	TRUE	13	CHARACTER is "-"
C)	. . . EXPR	TRUE	16	Call TERM
	. . . . TERM	TRUE	26	Call FACT
	. . . . . FACT	TRUE	49	CHARACTER is LETTER
)	. . . . TERM	TRUE	28	Back to TERM, start WHILE-DO
	. . . . TERM	FALSE	41	Tests for "*", "/" failed
	. . . . TERM	TRUE	43	But that's OK
	. . . EXPR	TRUE	5	Another WHILE-DO try
	. . . EXPR	FALSE	18	Failed this time
	. . . EXPR	TRUE	20	But that's OK
	. . FACT	TRUE	59	Where we left off in FACT
	. . FACT	TRUE	60	Character is ")"
ϵ	. . FACT	TRUE	68	Returns TRUE
	. TERM	TRUE	28	Try another WHILE-DO
	. TERM	FALSE	41	Failed
	. TERM	TRUE	43	But that's OK
	EXPR	TRUE	5	Try another WHILE-DO
	EXPR	FALSE	18	Didn't work
	EXPR	TRUE	20	But that's OK

Figure 4.6. Trace of program of figure 4.5 for the string "A*(B−C)".

4. Suppose code is emitted upon scanning the operators "+", "*", etc., as in the following table. What kind of target machine would support the emitted code? Give some examples of expressions and the emitted code.

operator	emitted instruction
+	ADD
−	SUB
*	MPY
/	DIV
letter	LOAD letter

5. Extend the program of figure 4.5 to include replacement statements of the form

<variable> = <expression>

4.4.1. Construction and Validation

We shall now show that a recursive descent parser can be constructed automatically from any context-free grammar. Unfortunately, it will not necessarily recognize the grammar's language, and may possibly recognize no language at all by getting into an endless loop. We must therefore develop a condition that a grammar yield a valid parser, and this requirement will turn out to be exactly the LL(1) condition for a grammar.

Construction

Let $G = \{N, \Sigma, P, S\}$ be a context-free grammar. We construct a recursive-descent parser M from G as follows:

1. For each nonterminal symbol A in N, construct a recursive type Boolean procedure A, with no formal parameters or local variables.

2. Let FLAG be a global logical variable, whose value is TRUE or FALSE.

3. Consider procedure A, for some nonterminal A, and the set of A productions:

$$A \rightarrow w_1$$
$$A \rightarrow w_2$$
$$\vdots$$
$$A \rightarrow w_n$$

Each of the strings w_1, w_2, etc. will become separate programs with one entry point and one exit point. Then procedure A is written as follows:

```
procedure A: boolean;
begin
   (program for w1);
   if FLAG then return(TRUE)
  else
   (program for w2);
   if FLAG then return(TRUE)
  else
   (program for w3);
        .
        .
        .
   (program for wn);
   return(FLAG)
end
```

(*Note*: We return the value of a typed procedure through the RETURN statement. This is not a Pascal convention, but one that we find useful for discussion purposes here).

4. The program for a right member w of some production depends on whether w is empty or not, and if not, the sequence of terminal and nonterminal symbols of which it is comprised.

(a) If w is empty, and the procedure name is A, then the program for w is written:

```
FLAG:=MEMBER(CHARACTER, FOLLOW(A))
```

where CHARACTER is the character currently under the read head, FOLLOW(A) is the FOLLOW set for A, and MEMBER is a Boolean procedure that returns TRUE if CHARACTER is in FOLLOW(A) and FALSE otherwise.

(b) Let $w = a_1a_2a_3 \ldots a_r$, where the a_i are terminal or nonterminal symbols and $r \geq 1$. Then the program for w is written:

```
if MATCH(a1) then
begin
   FLAG:=TRUE;
   if not MATCH(a2) then ERROR;
   if not MATCH(a3) then ERROR;
```

```
        .
        .
        .
    if not MATCH(a_r) then ERROR
end
else FLAG:=FALSE
```

where MATCH(X) depends on whether X is a terminal or a nonterminal symbol. If X is nonterminal, then MATCH(X) is simply X, i.e. a call of procedure X. If X is terminal, then MATCH(X) is a function call COMPARE(X), where function COMPARE is:

```
procedure COMPARE(X: char): boolean;
begin
    if X=CHARACTER then
    begin
        NEXTCHAR;
        return(TRUE)
    end
    else return(FALSE)
end
```

Procedure NEXTCHAR fetches the next character, placing it in variable CHARACTER. Some special symbol must be used to indicate the end of the string, disjoint from the alphabet set.

ERROR is an error routine; when called, the machine will block and a syntax error is reported at the position of the read head in the input string.

5. Given some input string z. The parser is invoked by setting the read head to the left-most symbol of z, placing it in CHARACTER, then calling procedure S, which corresponds to the start symbol in G. Then if the grammar G is LL(1), procedure S returns TRUE if z is in L(G) and FALSE otherwise.

Example. Consider the productions

$$E'' \rightarrow T'\ E''$$
$$E'' \rightarrow \epsilon$$

which appear in the LL(1) grammar given in figure 4.3. They may be converted to the function E″ given below, by following the above rules:

```
procedure E'': boolean;
begin
   if T' then
   begin
      FLAG:=TRUE;
      if not E'' then ERROR
   end
   else FLAG:=FALSE;
   if FLAG then return(TRUE)
   else
   FLAG:=MEMBER(CHARACTER, FOLLOW(E''));
   return(FLAG)
end
```

Now consider the productions

$$F \rightarrow a$$
$$F \rightarrow (E)$$

which also appear in figure 4.3. These may be converted to the procedure F given below:

```
procedure F: boolean;
begin
   if COMPARE("a") then
   begin
      FLAG:=TRUE
   end
   else FLAG:=FALSE;
   if FLAG then return(TRUE)
   else
   if COMPARE("(") then
   begin
      FLAG:=TRUE;
      if not E then ERROR;
      if not COMPARE(")") then ERROR
   end
   else FLAG:=FALSE;
   return(FLAG)
end
```

These procedures are more complicated than they need to be. They may easily be rewritten to improve their form and efficiency. It is also possible to

devise a more elaborate constructor algorithm that examines larger contexts in the extended grammar derivation tree, and to choose more appropriate control structures for the parser. Despite these objections, this construction is useful as a means of proving correctness and demonstrating the relation of recursive descent to LL(1) parsers. It will be less easy to do for the extended recursive descent system described later.

Exercises

1. Finish the parser based on figure 4.3, and implement it on some computer. Try a variety of valid and invalid strings.

2. Construct an equivalent LL(1) grammar for the following context-free grammar and construct a recursive descent parser from it (the goal symbol is <stmt>):

```
<stmt> ::= EXEC
         | BEGIN <decl-list> ; <stmt-list> END
<stmt-list> ::= <stmt-list> ; <stmt>
         | <stmt>
<decl-list> ::= <decl-list> ; DECL
         | DECL
```

3. Construct a parser for the grammar

$$S \rightarrow Abc \mid aAcb$$
$$A \rightarrow \epsilon \mid b \mid c$$

We have shown earlier that this grammar is not LL(1). Find a string for which the parser fails and determine just why and how the parser fails.

Grammar Validation

The following two examples show that the parser constructed from a grammar may fail. Neither grammar given below is LL(1), but the two grammars illustrate the two kinds of failure that a recursive descent parser will exhibit for a non-LL(1) grammar.

Failure Example 1. Consider the grammar

$$S \rightarrow S0 \mid 1$$

which yields strings in the regular expression $1\{0\}$. The parser constructed for this grammar consists of a single procedure, as follows:

```
procedure S: boolean;
begin
   if S then
   begin
      FLAG:=TRUE;
      if not COMPARE("0") then ERROR
   end
  else FLAG:=FALSE;
   if FLAG then return(TRUE)
  else
   if COMPARE("1") then
   begin
      FLAG:=TRUE
   end
  else FLAG:=FALSE;
  return(FLAG)
end
```

By tracing the procedure for a simple example, e.g., the string "10", we soon encounter a problem. Upon calling S, S immediately calls itself! There is no movement of the read head or any conditional testing. These calls will continue until the return address stack space is exhausted in the computer, causing a failure of the parser regardless of the input string.

It is clear from the grammar that the difficulty lies in the left-recursive production $S \rightarrow S0$. It is also clear that any left-recursion, whether simple or not, will cause a failure of the system on some input string.

This difficulty cannot be cured by reordering the productions, i.e., writing the grammar

$$S \rightarrow 1 \mid S0$$

to yield the following program, which scans the leading "1" nicely, but then fails to scan any subsequent "0"s. Furthermore, given any string whose first symbol is not "1", the program again falls into an endless sequence of recursive calls with no movement of the read head:

```
procedure S: boolean;
begin
   if COMPARE("1") then
   begin
      FLAG:=TRUE
```

```
    end
    else FLAG:=FALSE;
    if FLAG then return(TRUE)
    else
    if S then
    begin
       FLAG:=TRUE;
       if not COMPARE("0") then ERROR
    end
    else FLAG:=FALSE;
    return(FLAG)
end
```

Failure Example 2. The following grammar also yields a parser which fails to recognize the grammar's language:

$$S \rightarrow A \perp$$
$$A \rightarrow B \mid C$$
$$B \rightarrow a\ C$$
$$C \rightarrow a\ a$$

This language consists of the two strings "aa⊥" and "aaa⊥". Note that this grammar contains no left-recursive productions. The reader is invited to construct the parsers for this grammar G_2 and also for a grammar G_2', in which the second production is

$$A \rightarrow C \mid B$$

Now consider these parsers, P2 and P2′, respectively. Given the string "aa⊥" in P2, procedure S is called, which calls A. A calls B (since B is written first in the alternation, it will be called first). Procedure B accepts the first character "a", then calls C. However, C expects two more characters "aa", but rather sees only "a⊥". It therefore fails, setting the flag FALSE, which is reported back to B. Then B reports FALSE to A, which then calls C as the other alternative. Again, C reports failure, since it expects "aa" and sees "a⊥". The failure of A is reported to S, which reports failure for the string. We therefore have a failure to accept a string in the language defined by the grammar.

An examination of the cause of this failure reveals that B should back up the read head on a failure. It scanned "a" before calling C and finding that C was in trouble. Rather than blindly reporting failure, it would seem that B should attempt to restore the read head position and try another alternative before reporting failure. Unfortunately, a read head backup cannot be

confined to one or two operations in general, but must be part of a general backtracking system.

We therefore conclude that the parser fails for grammar G_2. Let us see if it also fails for grammar G_2', in which the A productions are

$$A \rightarrow C \mid B$$

This time, consider the input string "aaa⊥". Procedure S first calls A, which calls C first. C accepts the suffix "aa", and reports TRUE to A, which reports TRUE to S. However, S now expects "⊥", but sees "a" instead. It therefore fails.

Here, we see that the difficulty lies in procedure S; it gave up too soon. Rather than report an error, it should tell procedure A to try another alternative, if it had any left. However, this again requires a generalized backtracking system. Recall, from chapter 2, that a general backtracking system requires an exponential time to parse certain sentences, or to detect a syntax error, and is therefore impractical.

Recursive Descent Validation

In this section, we show that if the grammar is LL(1), then the recursive descent parser will recognize exactly the language of its basis grammar.

We assert that, if the grammar G is LL(1), then:

1. Every procedure A will halt in finite time on any input string.
2. If $S \Rightarrow^* xAy \Rightarrow xwy$, then procedure A will accept w within the input list wy. By *accept* we mean that A will return TRUE, having advanced the read head exactly through string w.
3. If procedure A accepts a string w, and $|w| > 0$, then $A \Rightarrow^* w$ in G.
4. If procedure A accepts the empty string with the input string y (without moving the read head, of course), then there exists an empty production $A \rightarrow \epsilon$ in G, and a derivation $S \Rightarrow^* xAz \Rightarrow xz$ such that FIRST(y) is in FIRST(z).

Corollary. If these four assertions are true, then it clearly follows that procedure S (where S is the grammar's start symbol) accepts exactly the language L(G) if G is LL(1). Exact acceptance requires that S reports TRUE and exactly scans every string in the language, and also that S reports FALSE or calls ERROR on every string not in the language.

We first show property 1, by showing that A returns in a finite number of operations.

Note first that no procedure contains an internal loop. There are simply alternative tests involving COMPARE operations, FOLLOW operations and calls on other procedures, and a finite number of these exist. We may therefore count a procedure call as an operation. Now a procedure A may call as many as $n-1$ other procedures without any movement of the read head, where n is the number of nonterminals in the grammar. If it calls n procedures, then one of these must be A, and this implies the existence of a derivation $A \rightarrow^* A\ldots$, and G cannot be LL(1). Each of these $n-1$ calls may call at most $n-2$ procedures, etc., yielding at most $(n-1)! = (n-1)\times(n-2)\times \ldots \times 2$ calls without moving the read head. Upon exhausting these calls, the read head must move, or A returns, or else ERROR is called.

We see that in at most $(n-1)!$ calls, procedure A must complete its operations. (In practice, A completes its task in a much smaller number of calls. It requires a time proportional to the length of the string it scans). QED

Now consider property 2. Let string w be such that $S \Rightarrow^* xAy \Rightarrow^* xwy$. We shall show that function A accepts w within xwy by induction on the number of derivation steps in $A \Rightarrow^* w$. Let the first step be

$$A \Rightarrow x_1x_1 \ldots x_n \Rightarrow^* w_1w_2 \ldots w_n$$

where $x_1 \Rightarrow^* w_1$, $x_2 \Rightarrow^* w_2$, etc., and $A \rightarrow x_1x_2 \ldots x_n$ is in P.
If $n=0$, then this is an empty production, $A \rightarrow \epsilon$. Since $w=\epsilon$ is within a sentential form xwy, CHARACTER is in FIRST(y) and is in the FOLLOW(A) set, since xAy is a sentential form. Therefore function A returns TRUE without moving the read head.

If x_1 is a terminal symbol, then $x_1 = w_1 =$ CHARACTER. Some COMPARE operation in A will succeed; furthermore, since G is LL(1), there can be exactly one such COMPARE function for this symbol x_1. Also if this COMPARE must compete with some function call B for acceptance of a string beginning with w_1, B must return false. If B returned TRUE, it would imply the existence of a derivation

$$B \Rightarrow^* w_1 \ldots$$

by the inductive hypothesis, and it further implies, by the way the function A is constructed, that $A \Rightarrow B\ldots \Rightarrow^* w_1\ldots$ and also $A \Rightarrow w_1\ldots$ directly, which is a violation of the LL(1) condition. We therefore conclude that there must exist exactly one COMPARE test for w_1, and that no competing procedure call can succeed. If a FOLLOW test exists, by the LL(1) condition, it cannot succeed either, else there would be a conflict between FOLLOW(A) and $A \rightarrow w_1\ldots$.

If x_1 is a nonterminal symbol, then there must be a CALL x_1 among the alternatives in the A procedure. Given that w is derivable from A, and w_1 derivable from x_1, by the inductive hypothesis, CALL x_1 must accept string w_1, reporting TRUE.

Once the first element x_1 is identified—it must be identified by one of the alternatives in procedure A—the remaining elements $x_2, x_3, \ldots, x_n$ must be scanned by COMPARE or CALL operations, else an ERROR results. A COMPARE will scan a character; the CALL x_i, by the inductive hypothesis, will scan string w_i and report TRUE. Therefore A accepts w in the context of the sentential form xwy. QED

Next suppose that $|w| > 0$, and that A accepts w through some alternative based on a production $A \rightarrow x_1 x_2 \ldots x_n$, where the x_i are terminal or nonterminal (property 3.) Now each of the x_i must accept a portion of w, by the inductive hypothesis, either through a COMPARE, a FOLLOW, or a procedure call operation, within A. Also the successful operations each move the read head through 0 or more positions in w, in the order $x_1, x_2, \ldots x_n$. Therefore by the inductive hypothesis $x_1 \Rightarrow^* w_1$, $x_2 \Rightarrow^* w_2, \ldots x_n \Rightarrow^* w_n$, where $w = w_1 w_2 \ldots w_n$. Therefore $A \Rightarrow^* w$.

Consider property 4. Assume that A accepts some string w where $|w| = 0$. Then w had to be accepted with no movement of the read head. This requires either an immediate acceptance through the FOLLOW test, or a sequence of calls culminating in a successful FOLLOW test. Suppose the former. Then CHARACTER is matched against some member of FOLLOW(A), which implies a derivation

$$S \Rightarrow^* xAy \Rightarrow xy$$

where CHARACTER is also in FIRST(y).

Now suppose that $w = \epsilon$ is accepted through some chain of calls. By induction on the number of procedure calls required to accept w, given that calls on procedures B, C, . . . , M are required in that order to accept w, then there must exist a production $A \rightarrow BC \ldots M$ and the derivation

$$A \Rightarrow BC \ldots M \Rightarrow^* \epsilon$$

QED

4.4.2. Extended Grammars

The construction process given in the previous section is not an effective substitute for an LL(1) table-driven parser. It requires a transformation of a production set in general in order to obtain an equivalent LL(1) production set, and once this is done, there is no advantage in writing a recursive descent program for the production set.

By using an extended grammar, a more aesthetically satisfying recursive descent parser may be automatically constructed. However, such a parser may also fail to recognize the language defined by the grammar and requires validation. The validation rules are more complicated to express, and require a set of tree structures. We shall develop the parser generation process and discuss validation without attempting proofs of the algorithms employed.

An extended grammar consists of a set of extended productions of the form

$$A \rightarrow w$$

where w is an *extended structure* (as next defined) and A is a nonterminal; for each nonterminal A, there is exactly one such production in the grammar.

An *extended structure* is a string consisting of terminals, nonterminals, and the meta-symbols |, (,), [,], {, and } as follows:

- ϵ (the empty string) is an extended structure.
- If $x \in \Sigma$ (i.e., a terminal symbol), then x is an extended structure.
- If $X \in N$ (i.e., a nonterminal symbol), then X is an extended structure.
- If S is an extended structure, then each of the following are extended structures:

  (S), meaning S itself.
  { S }, *closure*, meaning zero or more concatenations of S.
  [S], *option*, meaning zero or one occurrence of S.

- If S_1 and S_2 are extended structures, then each of the following are extended structures:

  $S_1 \mid S_2$, *alternation*, meaning a choice of S_1 or S_2.
  $S_1\ S_2$, *concatenation*, with the usual meaning.

It is clear from the above structure rules that the right member w of each extended production $A \rightarrow w$ is a regular expression, except that both terminal and nonterminal symbols are employed in the expression. We have also introduced a new unary operator, *option*.

Example. Grammar G_0 may be written as an extended grammar as follows. The productions $E \rightarrow T$, $E \rightarrow E + T$, and $E \rightarrow E - T$ define a structure for E that looks like this:

$$E \rightarrow T\ \{\ (\ +\ |\ -\ )\ T\ \}$$

That is, any expression consists of a *term* followed by any number of $+$*term* or $-$*term* elements, including none. Note that E no longer appears in the right-hand part of the extended production.

An equivalent way of expressing these productions is

$$E \rightarrow T\ \{\ (\ +\ T\ )\ |\ (\ -\ T\ )\ \}$$

The equivalence is apparent from the distributive law of concatenation over alternation.

Similarly, the productions T → F, T → T * F, T → T / F define a structure for T that looks like this:

$$T \rightarrow F \{ (* \mid /) F \}$$

Finally, the productions F → (E) and F → a define the structure:

$$F \rightarrow lp\ E\ rp \mid a$$

where *lp* and *rp* stand for the terminal symbols "(" and ")", respectively. Since "(" and ")" are metasymbols, we cannot permit these as terminal symbols.

We therefore have the extended grammar G_0':

$$\begin{aligned} E &\rightarrow T \{ (+ \mid -) T \} \\ T &\rightarrow F \{ (* \mid /) F \} \\ F &\rightarrow lp\ E\ rp \mid a \end{aligned}$$

4.4.3. Construction and Validation from an Extended Grammar

Given an extended grammar G = (N, Σ, P, S), we may construct a recursive descent parser from it by following the plan given in figure 4.7.

In using figure 4.7, an extended structure is best represented by a tree, containing *concatenation*, *alternation*, *closure*, and *option* internal nodes, along with terminal and nonterminal leaf nodes. Figure 4.7 then specifies the recursive descent program components developed from such a tree in a preorder scan. For example, the structure

$$T\{(+ \mid -)T\}$$

is decomposed as follows:

- the concatenation of T with "{(+ | −)T}".
- the closure "{(+ | −)T}" of "(+ | −)T".
- the concatenation of "(+ | −)" with T.
- the parenthesized structure "+ | −".
- the alternation of + with −.

The corresponding tree structure is given in figure 4.8.

The concatenation of T with "{(+ | −)T}" yields the program:

```
(program for T);
if FLAG then
begin
   (program for "{(+ | −)T}" )
end
```

For each production A → w

Construct a recursive procedure as follows:

```
procedure A;
begin
  (program based on structure w)
end
```

For a nonterminal structure X

Construct the procedure call:

```
X     (e.g. call procedure X)
```

For a parenthesized structure (w)

Construct the program sequence:

```
(program for w)
```

For an option [w]

Construct the program sequence:

```
(program based on w);
FLAG: = TRUE;
```

For a terminal symbol structure "a"

Construct the statement:

```
if CHARACTER = 'a' then
begin
  NEXTCHAR; (get the next character)
  FLAG: = TRUE
end else FLAG: = FALSE
```

For a concatenation structure w1 w2

Construct the program sequence:

```
(program for w1);
if FLAG then
begin
  (program for w2)
end
```

Figure 4.7. Construction rules for a recursive descent parser, based on an extended grammar representation of the language.

For a closure structure { w }

Construct the program sequence:

```
FLAG: = TRUE;
while FLAG do
begin
  (program based on w)
end;
FLAG: = TRUE;
```

For an alternation structure w1 | w2

Construct the program sequence:

```
(program based on w1);
if not FLAG then
begin
  (program based on w2)
end
```

Figure 4.7. (cont'd.)

It in turn expands into the program:

```
T;    {call procedure T}
if FLAG then
begin
   FLAG:=TRUE;
   while FLAG do
   begin
      (program for "(+ | −)T" )
   end;
   FLAG:=TRUE
end
```

The program for "(+ | −)T" is, at the outermost level, a concatenation, and, further down, an alternation of two terminal symbols:

```
(program for "+ | −" );
if FLAG then
begin
   T
end
```

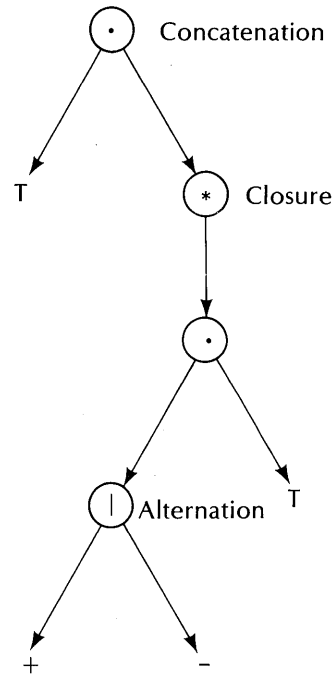

Figure 4.8. Tree for extended grammar structure T{(+ | −)T}.

Finally, the program for "+ | −" is:

```
if CHARACTER="+" then
begin
  NEXTCHAR;
  FLAG:=TRUE
end else FLAG:=FALSE;

if not FLAG then
begin
  if CHARACTER="−" then
  begin
      NEXTCHAR;
      FLAG:=TRUE
  end else FLAG:=FALSE
end
```

Putting all these together yields the program:

```
T;
if FLAG then
```

```
begin
   FLAG:=TRUE;
   while FLAG do
   begin
      if CHARACTER="+" then
      begin
         NEXTCHAR;
         FLAG:=TRUE
      end else FLAG:=FALSE;
      if not FLAG then
      begin
         if CHARACTER="-" then
         begin
            NEXTCHAR;
            FLAG:=TRUE
         end else FLAG:=FALSE
      end;
      if FLAG then
      begin
         T
      end
   end;
   FLAG:=TRUE
end
```

We need only embed the preceding statements in the procedure block:

```
procedure E: boolean;
begin
  (above program)
end
```

and we have a procedure that is equivalent to the EXPR procedure in figure 4.5. Its arrangement is slightly different, but the recognition logic is the same.

Finding FIRST and FOLLOW Sets for an Extended Grammar

A parser constructed from an extended grammar by the rules of figure 4.7 may or may not recognize the language of the grammar. In general, it will if the grammar is LL(1). However, we must formulate a more general LL(1) condition for an extended grammar. We can do so most conveniently if the grammar is represented as a set of trees, one tree for each nonterminal. The necessary tree structure is defined by the following rules:

1. For every nonterminal A in an extended grammar, construct a syntax tree. The root node will carry symbol A. The internal nodes other than the root will carry the grammar operations for alternation, concatenation, closure and option. The leaf nodes will carry a terminal or nonterminal symbol of the grammar or ϵ. Each tree represents a production in the extended grammar. A set of trees for grammar G_0' is given in figure 4.9. The symbols {., *, @, |} represent concatenation, closure, option, and alternation, respectively.

2. Each node will also carry two lists of terminal symbols, representing FIRST and FOLLOW associated with the node.

3. Construct a FIRST list for each node N as follows:

(a) If node N is associated with the nonterminal A, then locate the tree rooted in A; let its root node be R, and add the list FIRST(R) to FIRST(N), and add FIRST(N) to FIRST(R). This rule ensures that every node associated with a nonterminal symbol A, including a root node, carries the same list of FIRST symbols.

(b) If N is associated with the tree root R, then add FIRST(CHILD(R)) to FIRST(N).

(c) If N is associated with ϵ or a terminal symbol x, then FIRST(N) = $\{\epsilon\}$ or $\{x\}$, respectively.

(d) If N is associated with a closure, or an option, then add FIRST(CHILD(N)) to FIRST(N), and add ϵ to FIRST(N).

(e) If N is associated with a concatenation, then add

FIRST(FIRST(LEFTCHILD(N)) FIRST(RIGHTCHILD(N)))

to FIRST(N). LEFTCHILD(N) is the left child node, and RIGHTCHILD(N) is the right child node of N.

(f) If N is associated with an alternation, then add

FIRST(LEFTCHILD(N)) $\cup$ FIRST(RIGHTCHILD(N))

to FIRST(N).

4. Repeat step 3 until no more symbols can be added to any of the FIRST lists. A finite number of operations is required to reach that point. since the trees are finite and there are a finite number of symbols that may appear in any of the lists.

5. Construct the FOLLOW list for each node N as follows:

(a) If N is the root node, associated with a nonterminal A, then add FOLLOW(V) to FOLLOW(N) for each node V in each of the trees associated with the nonterminal A, and add FOLLOW(V) to FOLLOW(N). This rule ensures that every node associated with some nonterminal A carries the same FOLLOW list.

(b) If N is associated with the grammar's goal symbol S, add ϵ to

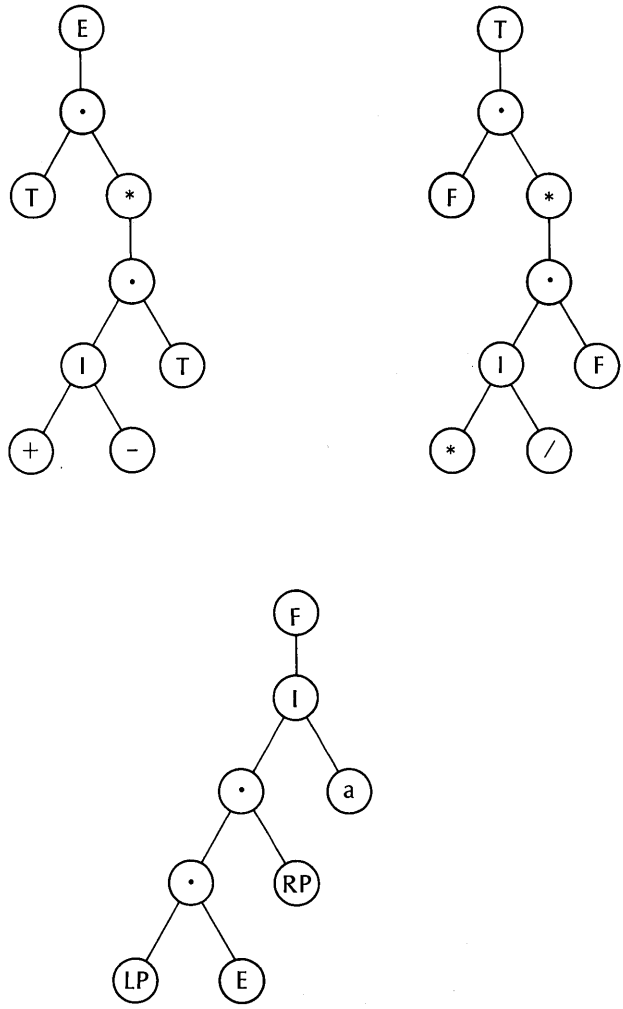

Figure 4.9. Syntax trees for grammar G_0'.

FOLLOW(N), and to the FOLLOW lists of all other nodes associated with S.

In the remaining rules, let F = PARENT(N), where N is not the root node:

(c) If F is a concatenation node, and N = LEFTCHILD(F), then add

FIRST(FIRST(RIGHTCHILD(F)) FOLLOW(F))

to FOLLOW(N).

(d) In all other cases, add FOLLOW(F) to FOLLOW(N).

6. Rule 5 is repeated until no more additions to the FOLLOW lists can be made.

The meaning of FIRST(X Y), where X and Y are lists, is

$$\begin{aligned} \text{FIRST(X Y)} &= (\text{X} - \{\epsilon\}) \cup \text{Y} && \text{if } \epsilon \text{ is in X, or} \\ &= \text{X} && \text{if } \epsilon \text{ is not in X.} \end{aligned}$$

Discussion

We shall not attempt a complete justification of these rules. That would require more development of extended derivations and extended sentential forms, etc. However, the notion of FIRST and FOLLOW is essentially that developed for simple grammars, except that both are lists of terminal symbols. A FIRST list is the set of all symbols that appear first in a sentence derivable from that node.

For example, if node N is an alternation, then its FIRST list should include FIRST(L) and FIRST(R), where L and R are its children, since a sentence derivable from N includes sentences derivable from L and from R. Similarly, if N is a concatenation, with children L and R, then the sentences of N are derivable from sentences of the form XY, where X is derivable from L and Y from R. Clearly, FIRST(N) includes FIRST(XY) = FIRST(L FIRST(R)).

If node N is a closure or option, then its FIRST includes FIRST(C), where C is its child, but also includes ϵ, since these structures may derive the empty string.

If node N is a terminal symbol, then clearly FIRST(N) contains only that symbol.

Finally, if node N is a nonterminal, it may be a root node, in which case FIRST(N) includes FIRST(C), where C is its child. We also ensure that all the nodes associated with a given nonterminal carry the same FIRST lists.

The FOLLOW rules essentially state that ϵ follows a goal symbol, and that FOLLOW lists propagate downward through the tree, except to left children of concatenation nodes. The FOLLOW set of a left child N of a concatenation is the FIRST list of N's right sibling, and if this list contains ϵ, also includes the FOLLOW of N's right sibling. Any FOLLOW members found for some nonterminal node V are copied to all the other nodes associated with V.

For example, consider the set of trees representing the extended grammar G_0':

$$\begin{aligned} &\text{E} \rightarrow \text{T } \{ \ (+ \mid -) \ \text{T}\} \\ &\text{T} \rightarrow \text{F } \{ \ (* \mid /) \ \text{F}\} \\ &\text{F} \rightarrow \textit{lp} \ \text{E} \ \textit{rp} \mid \text{a} \end{aligned}$$

The syntax trees for G_0' are shown in figure 4.9.

Figure 4.10 shows the same trees decorated with the FIRST lists. These lists were constructed from rule 3 above. A convenient place to start is some terminal node, for example *lp*, *rp*, or "a" in the F tree, using rule 3(c). The concatenation nodes in tree F clearly carry *lp* by rule 3(e) and the observation that the left children do not contain ϵ. The alternation node in tree F carries the union of the FIRST lists of its children, "a" and *lp*, by rule 3(f), and F carries its child's FIRST list by rule 3(b). The F list may now be entered in the other trees, in particular the T tree, which is similarly built up.

A partial FOLLOW list for the trees of figure 4.9 are shown in figure 4.11. For these, it is convenient to begin by adding ϵ to the goal symbol nodes E, then looking for concatenation nodes in which the right child is terminal or contains a nonempty FOLLOW list. Any additions to a FOLLOW list must then be propagated down the tree by rule 5(d). Thus the left T node in the E tree carries $\{+, -, \epsilon\}$ since these are in the FIRST set of its right sibling, a closure node. Similarly, $\{*, /, \epsilon\}$ are in the FOLLOW list of the left F node in the T tree. Since ϵ is in FOLLOW for E, it propagates down the E tree along its right. It cannot propagate to the left child of a concatenation node, however. Similarly, the set $\{+, -, \epsilon\}$ associated with any T node is propagated down the right side of the T tree, and eventually is added to the FOLLOW set of F. (We have not yet added to F the $\{*, /\}$ symbols found in the left node of the T tree.) Then the set $\{+, -, \epsilon\}$ found for F so far is also propagated down the F tree. However, these do not decorate the E node, since a concatenation node lies two levels above it.

Through repetitive applications of rule (5), the final FOLLOW lists appear as shown in figure 4.12. Note that the terminal nodes are also decorated. As a check on our work, these lists appear to be reasonable in terms of the known contexts of arithmetic operators. Any symbol in $\{*, /, +, -, \epsilon\}$ can follow an operand, as is clear from the following examples:

a*a a/a a+a a−a a

Similarly, either symbol in {a, (} can follow any of the arithmetic operators $\{+, -, *, /\}$ as is clear from these examples:

a*a a+(a−a)

Finally, a right parenthesis may be followed by any symbol in $\{*, /, +, -, \epsilon,)\}$ as is clear from the following examples:

(a)*a (a)/a (a)+a (a)−a (a+a) ((a+a))

The FOLLOW lists for the nonterminal symbols are less obvious; these lists and the lists for the internal nodes depend strongly on the grammar.

Parser Validation

A recursive descent parser constructed from an extended grammar is valid if the following two conditions on the syntax trees of the grammar are met:

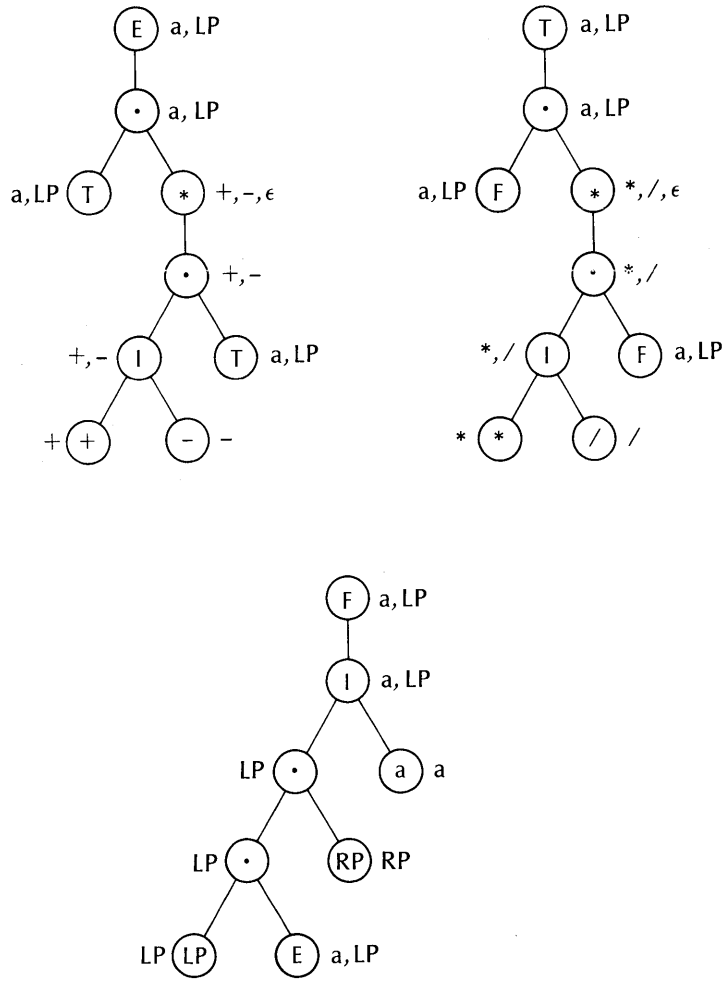

Figure 4.10. FIRST lists of the trees of figure 4.9.

1. For every closure or option node N,

$$\text{FIRST}(\text{FIRST}(\text{CHILD}(N))\ \text{FOLLOW}(N)) \cap \text{FOLLOW}(N) = \varnothing.$$

2. For every alternation node N,

$$\text{FIRST}(\text{FIRST}(\text{LEFTCHILD}(N))\ \text{FOLLOW}(N)) \cap$$

$$\text{FIRST}(\text{FIRST}(\text{RIGHTCHILD}(N))\ \text{FOLLOW}(N)) = \varnothing$$

We shall not attempt a formal proof of these assertions, but will discuss

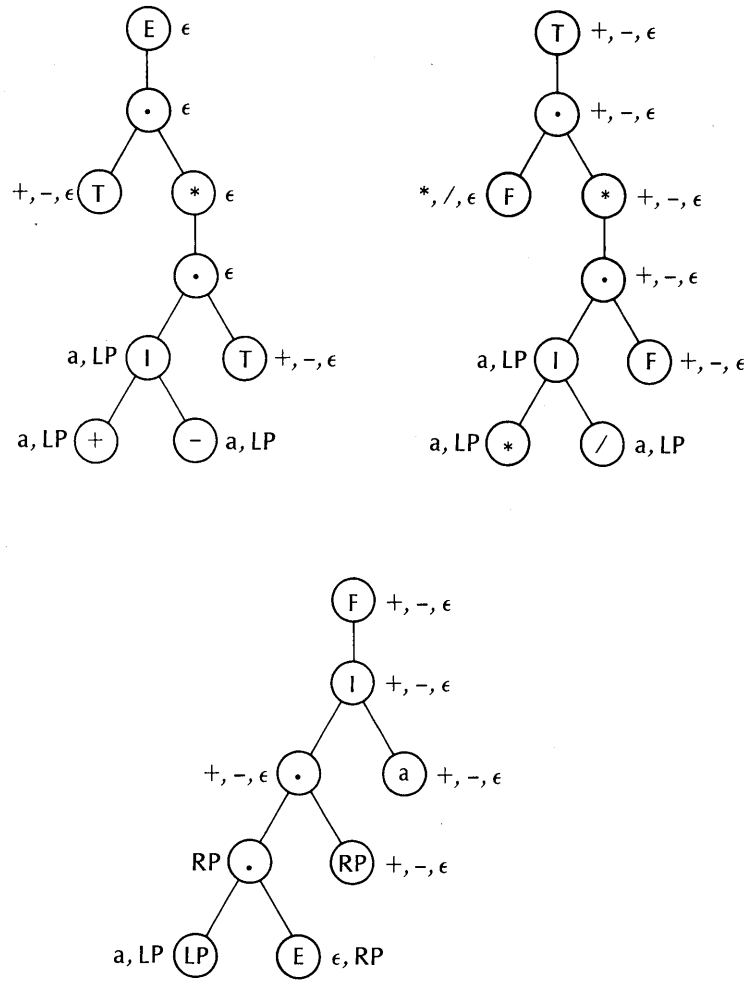

Figure 4.11. Partial FOLLOW lists for the trees of figure 4.9.

them informally. Note that these conditions are similar to the LL(1) conditions for a simple grammar. In an extended grammar, if the recursive descent parser fails to accept some input string, it will fail through either accepting some portion of the wrong alternative or by accepting a portion of a closure or option when the following string should instead have been accepted.

Rule 2 above guarantees that the parser can never choose the wrong alternative upon examining the next input symbol, since a pair of alternatives can only derive strings starting with disjoint symbols.

Rule 1 guarantees that when the parser attempts to recognize a closure or option, a following string cannot carry a conflicting FIRST symbol. That is,

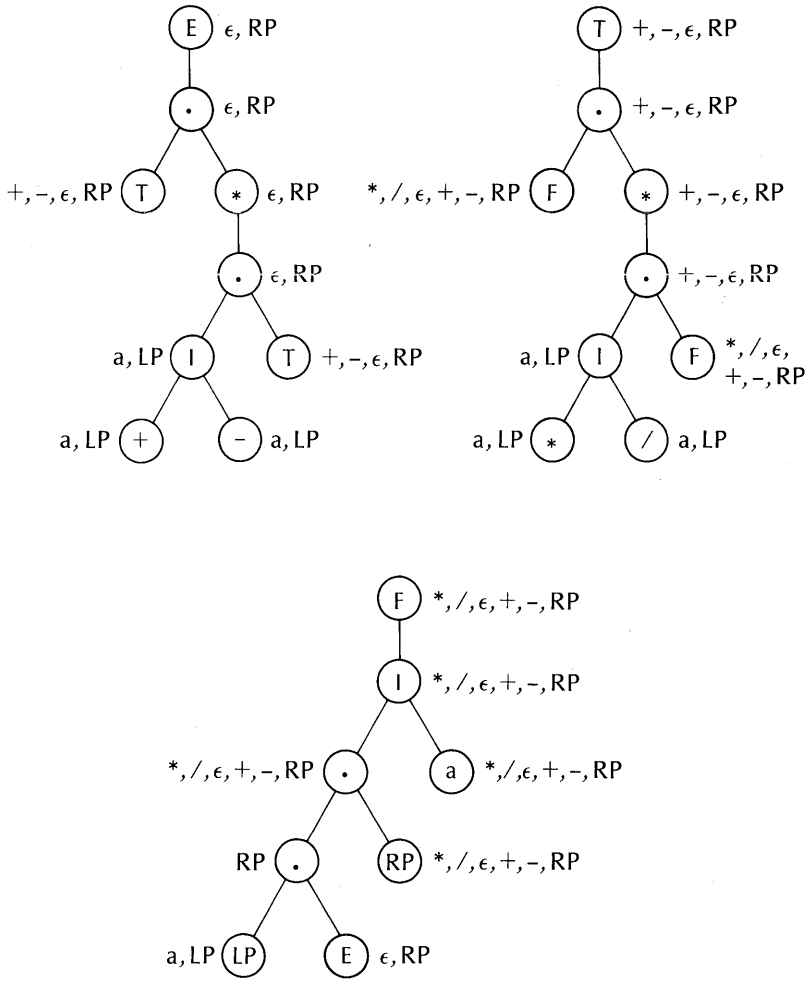

Figure 4.12. Complete FOLLOW lists for the trees of figure 4.9.

consider the form

$$\ldots\{x\}y$$

Rule 1 guarantees that the strings derivable from x cannot contain any leading symbol in common with the strings derivable from y. As a special case, x must not be able to derive an empty string ϵ, since then the parser cannot possibly determine whether a closure operation on ϵ is required or not. This case is covered by the rule, since if M (in the rule) can derive ϵ, then FIRST(M FOLLOW(N)) includes FOLLOW(N). Hence, if the parser attempts to accept a string derivable from x, and succeeds on the first symbol, then it cannot succeed on any first symbol in a string derivable from y. Also, if the

parser can scan a first symbol in a string derivable from y, then it must fail to accept any string in x, by failing on the first symbol in x.

Consider the decorated trees of Figs. 4.10 and 4.12. The alternation nodes in all three trees clearly satisfy the second condition, for example, in the E tree, the alternation node's left child carries $\{+\}$ and its right child carries $\{-\}$. Similarly, in the F tree, the children of the alternation node carry the disjoint sets $\{(\}$ and $\{a\}$.

The FIRST set of the child of the closure node in the E tree is $\{+, -\}$, while the FOLLOW set of the closure node is $\{\epsilon,)\}$. Similarly, the closure node in the T tree satisfies the validation condition. We conclude that a recursive descent parser constructed from these trees will correctly recognize the language of its extended grammar, which is essentially the parser given in figure 4.5.

Extended Grammar Represented as FSA

The structures in the right members of the productions of an extended grammar have the form of a regular expression, except that the machine transitions may be terminal or nonterminal transitions, and we will have a family of machines. Let us explore this matter further.

Construct a reduced DFSA for each of the right members of the extended productions in some grammar G. We will then have a machine M(A) for every nonterminal A in the grammar. Assume that the grammar is LL(1), i.e., it satisfies the validation conditions given in the previous section. Then we embed the machine in a recursive procedure as follows:

```
procedure A;
begin
   var STATE: integer;  {the current state}
   STATE:=0;      {the start state}
   {program to execute machine M(A)}
   FLAG:=if SUCCESS then TRUE else FALSE;
   return
end
```

The variable STATE is a local variable and is therefore stacked upon calling any other procedure, then unstacked upon returning from it. STATE indicates the state of machine M(A), which may be organized as a program or as an interpreted table. If organized as a table, then only one interpretative procedure for all the machines need be utilized, provided that it too, is recursive.

The machine transitions are of two kinds: a terminal transition, and a nonterminal transition. A terminal transition results in accepting the present symbol in the machine's present state, or reporting failure. If the terminal is

accepted, the read head is advanced one symbol; otherwise it is not moved.

On a nonterminal transition, the machine simply calls another machine by stacking its state and calling the appropriate procedure. The called machine either reports success or failure. If success, then this machine may continue, otherwise it must report failure.

A given machine continues until it must report failure or until it has reached a halt state and is unable to scan the next symbol; in this latter case it may report success.

The value of this approach lies in the attractiveness of an extended grammar in describing a language. For example, an extended grammar is closely related to a syntax graph, with which a number of computer languages are defined. The size of the parser system will likely be considerably smaller than if it were coded as a recursive descent system. However, it may be more difficult to attach semantic operations to the machine transitions, since these may no longer be clearly related to the extended grammar structure operations.

4.5. Bibliographical Notes

LL(k) grammars were first defined by Lewis and Stearns (Lewis [1968]). The theory of LL(k) grammars and their relation to deterministic parsers was extensively developed by Rosenkrantz and Stearns (Rosenkrantz [1970b]). Other papers on LL(k) grammar theory are Aho [1972a], vol. I, chapters 5 and 8; Kurki-Suonio [1969], and Griffiths [1974b]. Recursive descent parsers and compiler systems have been considered by a large number of authors. They form the basis of many so-called *compiler-writer* systems. A representative set of papers is Irons [1961], Metcalfe [1964], Feldman [1968], Gries [1971] (chapter 4), Aho [1972a] (vol. I, chapter 6), Griffiths [1974c], and Wirth [1976c].